



Cairo University

Faculty of Computers & Artificial Intelligence

Operations Research & Decision Support Department

Predictive Maintenance Optimization for Aircraft Engines

The Graduation Project Submitted to
The Faculty of Computers and Artificial Intelligence,
Cairo University

In Partial Fulfillment of the Requirements
for the Bachelor's Degree

In

Operations Research and Decision Support

**Under Supervision of
Dr. Tarek Abou-El Enien**

Cairo University

July/2026



Cairo University

Faculty of Computers & Artificial Intelligence

Operations Research & Decision Support Department

Predictive Maintenance Optimization for Aircraft Engines

Submitted by

Mariam Muhammed	20220332
Hager Mahmoud Masry	20221236
Samia Reda Mohamadi	20231240
Aya Sayed Mohamed	20220077

**Under Supervision of
Dr. Tarek Abou-El Enien**

**Cairo University
July/2026**

ABSTRACT

Unplanned failures of aircraft engines are the single most costly event in an airline's maintenance operation, and the most disruptive to its schedule. This project builds a complete, data driven system that anticipates such failures and plans maintenance around them for a fleet of one hundred turbofan engines drawn from NASA's CMAPSS FD001 benchmark.

The work has two connected halves. In the first, an XGBoost regression model estimates the Remaining Useful Life (RUL) of every engine from its sensor history; on the FD001 test fleet, the model reaches a root mean square error of 14.23 cycles, matching a published convolutional neural network while remaining far simpler to train and interpret. In the second half, the predicted RUL is fed into an exact Integer Linear Programming (ILP) model that produces a cost optimal, capacity feasible maintenance schedule and ranks the engines by urgency. A safety deadline constraint, tied to the predictor's own error, is built into the model so that no engine is ever scheduled beyond the point at which operating it becomes unsafe.

The model is solved to global optimality with the open source HiGHS solver, verified against a hand solved example and an independent spreadsheet check, and delivered through an interactive Streamlit dashboard that carries a user from raw data to an exported schedule. On the full fleet, the optimal plan costs \$1,379,000, with ninety two engines serviced preventively and eight routed to corrective maintenance. The report closes by identifying the organizations, including airlines, maintenance providers, engine manufacturers, defense operators, lessors, and regulators, that stand to benefit from the approach.

DECLARATION

We hereby declare that our dissertation is entirely our work and genuine / original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade, and it may result in withdrawal of our bachelor's degree.

Group members:

Name

Signature

Mariam Muhammed Hashem

Hager Mahmoud Masry

Samia Reda Mohamadi

Aya Sayed Mohamed

PLAIGRISM CERTIFICATE

This is to certify that the project entitled “**Predictive Maintenance Optimization for Aircraft Engines**”, which is being submitted here for the award of the “**Bachelor of Computer and Artificial Intelligence Degree**” in “**Operations Research and Decision Support**”. This is the result of the original work by **Mariam Muhammed, Samia Reda, Aya Sayed, Hager Mahmoud** under my supervision and guidance. The work embodied in this project has not been done earlier for the basis of award of any degree or compatible certificate or similar title of this for any other diploma/examining body or university to the best of my knowledge and belief.

Turnitin Originality Report

Processed on:

ID:

Word Count:

Similarity Index

Similarity by Source

Internet Sources:

Publications:

Student Papers:

Date:

Supervisor Name: *Dr. Tarek Abu Eleinen*

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our supervisor, Dr. Tarek Abu Eleinen, for his invaluable guidance, continuous support, encouragement, and constructive feedback throughout the development of this project. His expertise and insightful suggestions have greatly contributed to the successful completion of our work.

We would also like to express our sincere appreciation to Eng. Nesma Mohamed for her continuous support, helpful guidance, and valuable feedback throughout this project. She was always willing to answer our questions and provide assistance whenever needed. Her encouragement and dedication helped us overcome many challenges and contributed significantly to the progress of our work.

We would also like to extend our heartfelt thanks to Eng. Afaf Hamada for her continuous assistance, valuable advice, and willingness to help whenever needed. Her support and encouragement were greatly appreciated throughout this project.

Finally, we would like to express our special thanks to Machine Learning Engineer Ahmed Amer for sharing his knowledge, providing technical support, and offering valuable insights that enhanced the machine learning aspects of this project.

We are deeply grateful to all of them for their time, patience, and continuous support. Their encouragement and guidance have been invaluable throughout this journey.

TABLE OF CONTENTS

Chapter 1: Introduction	14
1.1 Introduction	15
1.2 Problem Domain	15
1.3 Problem Statement	16
1.4 Proposed System	17
1.4.1 Aims and Objectives.....	18
1.4.2 Proposed System Features.....	18
1.5 Development Methodology.....	18
1.6 Resource Requirements.....	19
1.7 Report Layout.....	19
Chapter 2: Background / Existing Work.....	21
2.1 Introduction	22
2.2 Overview of the Project.....	22
2.3 Limitations of the Project	22
2.4 Innovations in the Project.....	23
2.5 Design of the Project	23
2.6 Existing Work	24
2.7 Theoretical Background	25
2.7.1 Predictive Maintenance Concepts	25
2.7.2 Remaining Useful Life (RUL) Estimation	25
2.7.3 Feature Engineering for Time-Series Sensor Data.....	25
2.7.4 Gradient Boosting and XGBoost.....	25
2.7.5 Benchmarking Against Deep-Learning Prognostics.....	26
2.7.6 Optimization and Maintenance Scheduling	26
2.8 Related Models and Techniques Used in Literature.....	26
2.9 Summary of Existing Datasets	26
2.10 Gap Analysis	27
2.11 Comparison with Previous Studies.....	27
2.12 Chapter Summary.....	27
Chapter 3: Data Analysis	28
3.1 Data Overview.....	29
3.2 Dataset Description	30
3.3 Data Preprocessing.....	30
3.4 Data Integrity Check	31
3.5 Data Cleaning.....	31
3.6 Data Exploration	32
3.7 Data Visualization	33
Chapter 4: Methodology.....	37

4.1 Introduction	38
4.2 RUL Prediction Methodology	38
4.2.1 Problem Formulation as a Regression Task	38
4.2.2 Target Preparation	39
4.2.3 Feature Engineering	39
4.2.4 Model Configuration	40
4.2.5 Evaluation Metrics	41
4.3 Benchmarking Methodology	41
4.3.1 Reference Model	41
4.3.2 Basis for Comparison	41
4.4 Optimization Methodology	42
4.4.1 Rationale	42
4.4.2 Decision Variables	42
4.4.3 RUL Driven Quantities	42
4.4.4 Objective Function	43
4.4.5 Base Constraints	43
4.4.6 Core Constraints of This Work	44
4.4.7 The Emergency Slot	44
4.4.8 Solution Method	45
4.4.9 Parameter Values and Sources	45
4.5 Chapter Summary	46
Chapter 5: Work Details	47
5.1 Introduction	48
5.2 Worked Example	48
5.2.1 Instance Definition	49
5.2.2 Step by Step Solution (Targets, Deadlines, Cost)	50
5.2.3 Verification Against the Solver	52
5.2.4 Independent Spreadsheet Check	53
5.3 System Implementation	54
5.3.1 Architecture (Pipeline Overview)	54
5.3.2 Technology Stack	55
5.3.3 Application Walkthrough	56
5.3.4 Dashboard	58
5.3.5 Export Functionality	59
5.4 Results and Discussion	60
5.4.1 RUL Prediction Results (XGBoost Performance)	60
5.4.2 Benchmarking Results (XGBoost vs CNN)	62
5.4.3 Optimal Maintenance Schedule	63
5.4.4 Priority Ranking	64
5.4.5 Key Performance Indicators (KPIs)	65
5.4.6 Visual Analysis	66
5.5 Chapter Summary	68
Chapter 6: Conclusions	69
6.1 Beneficiary Organizations	70
6.1.1 Airlines and Fleet Operators	71
6.1.2 MRO Providers	71

6.1.3 Engine OEMs.....	72
6.1.4 Military and Defense Aviation	72
6.1.5 Aircraft Lessors.....	73
6.1.6 Aviation Regulators and Safety Authorities	73
6.2 Conclusion.....	74
6.3 Future Work	76
References	79

LIST OF FIGURES

Figure 1.1: Turbofan engine sensor data represented as time-series.....	16
Figure 1.2: Impact of unexpected engine failure on operational cost and delay	17
Figure 1.3: Overall framework of the proposed predictive maintenance system, from raw sensor data to an exported maintenance schedule.....	18
Figure 2.1: Proposed architecture from raw CMAPSS sensor data to an exported maintenance schedule	24
Figure 3.1: Sample from the data.....	29
Figure 3.2: Engine Lifespan Distribution	33
Figure 3.3: Sensor Variance Ranking	33
Figure 3.4: RUL Distribution Before/After NASA Capping.....	34
Figure 3.5: Engine #1 Multi-Sensor Degradation Trajectory	35
Figure 3.6: Sensor Correlation Matrix (Post-Cleaning Feature Selection	36
Figure 4.1: Contribution of the three engineered feature groups to model accuracy, measured by grouped permutation importance. The degradation delta group dominates the other two groups.....	40
Figure 5.1: Illustrative timeline of the five engines in the worked example, ordered by predicted Remaining Useful Life.	50
Figure 5.2: Workflow of the target, deadline, and cost derivation applied to each engine in the worked example.	51
Figure 5.3: Reconstructed solver console output confirming an optimal objective value of sixty five for the worked example instance.	52
Figure 5.4: Independent spreadsheet solution of the worked example, showing the given data block, the step by step target, deadline, scheduled period, earliness, and cost columns, the resulting total cost of sixty five, and the two feasibility checks performed beneath the main table.	53
Figure 5.5: End to end architecture of the predictive maintenance pipeline, from the CMAPSS FD001 dataset to the exported schedule and dashboard.....	54
Figure 5.6: Illustrative mockup of the data loading screen of the predictive maintenance dashboard.	56
Figure 5.7: Illustrative mockup of the XGBoost training and evaluation screen of the dashboard.	57
Figure 5.8: Illustrative mockup of the optimization parameter configuration screen of the dashboard.	57
Figure 5.9: Representative dashboard view, showing the key performance indicator tiles above the schedule map and workshop load chart.	58
Figure 5.10: Illustrative mockup of the export screen of the predictive maintenance dashboard.....	59
Figure 5.11: Predicted versus true Remaining Useful Life on the FD001 test fleet. Points lying close to the diagonal indicate accurate predictions.	60
Figure 5.12: Distribution of the Remaining Useful Life prediction error across the FD001 test fleet.	61
Figure 5.13: Predicted and true Remaining Useful Life across the test fleet, with engines sorted by true remaining life. ..	61
Figure 5.14: Root mean square error comparison between the reference CNN model and the XGBoost model developed in this project, shown alongside the CNN figures for the harder CMAPSS subsets for context.....	62
Figure 5.15: Gantt style timeline reconstructed from the exported schedule, showing the twelve highest priority preventive cases and the eight emergency cases from Table 5.9, coloured by maintenance mode.	64

Figure 5.16: Bar chart of the ten highest priority engines in the fleet, ranked by scheduled maintenance period and shaded by predicted Remaining Useful Life. 65

Figure 5.17: Maintenance mode split across the fleet, shown as a proportion of engines on the left, and the corresponding total cost attributed to each maintenance mode on the right. 67

Figure 5.18: Workshop load per planning period across the full scheduling horizon, plotted against the fixed capacity limit of three engines per period. 67

Figure 6.1: The predictive maintenance and optimization system at the center of its wider beneficiary ecosystem, spanning operators, maintenance providers, manufacturers, defense aviation, lessors, and regulators. 70

Figure 6.2: Roadmap of future work directions, organised into near term methodological extensions, data and modelling extensions, optimization extensions, and cross cutting deployment work. 76

LIST OF TABLES

Table 4.1: Main configuration settings adopted for the XGBoost regression model.....	41
Table 4.2: The three core constraints introduced in this work, and the dimension of the scheduling problem that each one bounds.	44
Table 4.3: Complete list of optimization model parameters, their adopted values, and the provenance of each value.....	45
Table 5.1: Parameters and predicted Remaining Useful Life values adopted for the five engine worked example.....	49
Table 5.2: Predicted Remaining Useful Life for the five engines used in the worked example.	49
Table 5.3: Derived target and deadline periods for the five worked example engines.	50
Table 5.4: Complete scheduling and cost calculation for the five worked example engines. Total cost equals sixty five...	51
Table 5.5: Comparison between the manually derived schedule and the schedule returned by the HiGHS solver for the worked example.	52
Table 5.6: Technology stack adopted for the implementation of the predictive maintenance system.....	55
Table 5.7: Prediction accuracy of the XGBoost Remaining Useful Life model on the FD001 test fleet.	60
Table 5.8: Prognostic accuracy, expressed as root mean square error in cycles, of the reference CNN model and the XGBoost model developed in this project. Lower values indicate better accuracy.	62
Table 5.9: Excerpt of the optimal maintenance schedule for the full fleet, showing the twelve highest priority engines and the eight engines routed to emergency maintenance. The complete one hundred engine ranking is produced directly by the application.	Error! Bookmark not defined.
Table 5.10: Operational key performance indicators for the optimal fleet wide maintenance schedule.	65
Table 6.1: Summary of the principal benefit derived by each beneficiary organisation from the predictive maintenance and optimization system.....	74

LIST OF ACRONYMS

CMAPSS	Commercial Modular Aero-Propulsion System Simulation
RUL	Remaining Useful Life
ILP	Integer Linear Program(ming)
XGBoost	Extreme Gradient Boosting
CNN	Convolutional Neural Network
RMSE	Root-Mean-Square Error
MAE	Mean Absolute Error
KPI	Key Performance Indicator
MRO	Maintenance, Repair and Overhaul
OEM	Original Equipment Manufacturer
GBDT	Gradient-Boosted Decision Trees

NOMENCLATURE

RUL_i	predicted remaining useful life of engine i (cycles)
Δ	safety margin (cycles), set to the predictor RMSE
ρ	operating cycles consumed per planning period
β	safety factor, $0 < \beta \leq 1$
t_i^{tgt}	target maintenance period of engine i
d_i	deadline period of engine i
C_t	workshop capacity in period t
c^m	base preventive-maintenance cost
p_e, p_l	earliness / lateness penalty per period
$c_{i,t}$	cost of servicing engine i in period t
p_g	emergency (corrective) penalty
B	maintenance budget
$a_{i,r}, R_{r,t}$	resource requirement / availability
$x_{i,t}$	binary decision: service engine i in period t
y_i	binary decision: route engine i to the emergency slot

Chapter1

Introduction

1.1 Introduction

Engine failures in aircraft can have serious consequences for safety, maintenance costs, and overall operational performance. Unexpected engine breakdowns not only pose risks to passenger safety but also disrupt flight schedules and increase repair expenses. Traditional maintenance approaches, such as fixed-interval inspections, often fail to account for variations in engine operating conditions and usage patterns, which can result in unnecessary maintenance or unforeseen failures.

With the advancement of data-driven technologies and artificial intelligence, predictive maintenance has emerged as an effective strategy to enhance the reliability and efficiency of aircraft operations. By leveraging historical sensor data, predictive maintenance enables the estimation of an engine's Remaining Useful Life (RUL), allowing maintenance activities to be planned proactively rather than reactively.

This project develops a complete, data-driven predictive maintenance system for a fleet of one hundred turbofan engines drawn from NASA's Commercial Modular Aero-Propulsion System Simulation (CMAPSS) FD001 benchmark. The work has two connected halves. In the first, an Extreme Gradient Boosting (XGBoost) regression model estimates the Remaining Useful Life of every engine from its sensor history, reaching an accuracy competitive with a published convolutional neural network (CNN) benchmark while remaining far simpler to train and interpret. In the second, the predicted RUL is fed into an exact Integer Linear Programming (ILP) model that produces a cost-optimal, capacity-feasible maintenance schedule, ranks the engines by urgency, and embeds a safety-deadline constraint so that no engine is ever scheduled beyond the point at which operating it becomes unsafe. The complete pipeline, from raw sensor data to an exported maintenance schedule, is delivered through an interactive dashboard.

1.2 Problem Domain

The operational reliability of aircraft engines is critical to the aviation industry. Engine failures can cause significant disruptions to flight schedules, increase maintenance costs, and pose potential safety hazards. Traditional maintenance approaches, such as fixed-interval inspections, often fail to account for variations in engine operating conditions, environmental factors, and usage patterns. This limitation can result in either unplanned engine failures or unnecessary maintenance, potentially compromising both safety and operational efficiency.

Aircraft engines generate multivariate sensor data over time, including temperature, pressure, rotational speed, and flow-rate measurements, as shown in Figure 1.1.

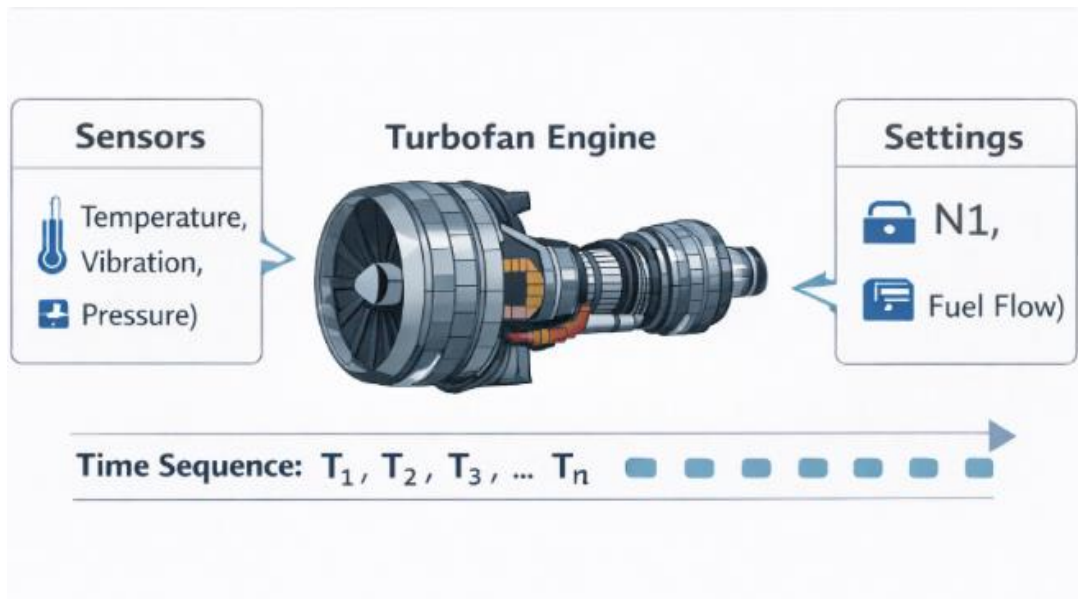


Figure 1.1: Turbofan engine sensor data represented as time-series

Predictive maintenance has emerged as an effective solution to these challenges by continuously monitoring engine health through historical sensor data. By estimating the Remaining Useful Life (RUL) of engines, maintenance activities can be scheduled proactively, reducing downtime, optimizing resource utilization, and improving overall operational performance.

This project addresses this domain by developing a predictive maintenance framework for the CMAPSS FD001 turbofan fleet. The project focuses on accurately estimating the RUL of each engine using a gradient-boosted regression model, and on generating a cost-optimal, capacity-feasible maintenance schedule that balances maintenance cost, workshop capacity, and operational safety.

1.3 Problem Statement

Aircraft engines are critical components whose failure can have serious consequences for safety, operational efficiency, and maintenance costs. Traditional maintenance strategies, such as fixed-interval inspections, do not adequately account for the variability in engine operating conditions and usage patterns. As a result, maintenance may be either unnecessary or insufficient, leading to unexpected engine failures, increased downtime, higher operational costs, and potential safety hazards.

As shown in Figure 1.2, unexpected engine failures result in significant operational costs and delays, which motivates the need for predictive maintenance approaches.

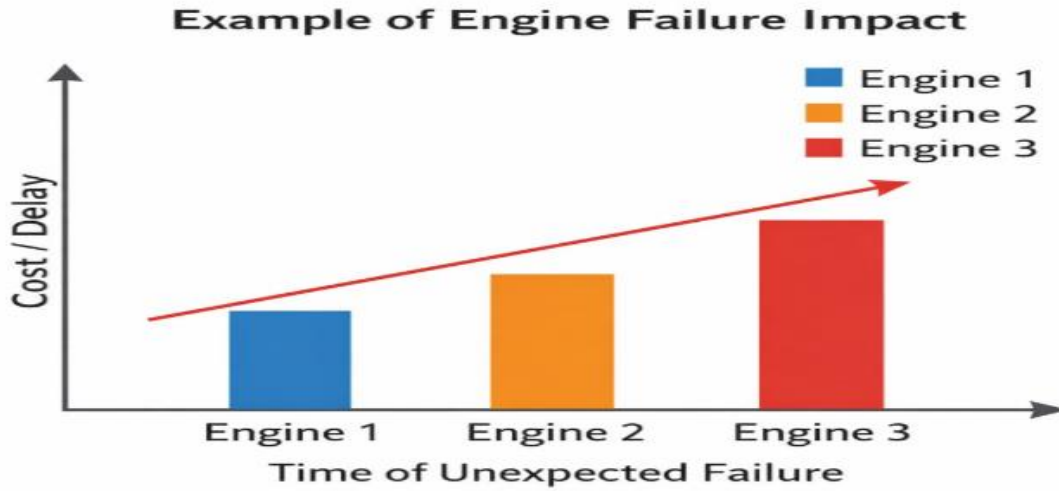


Figure 1.2: Impact of unexpected engine failure on operational cost and delay

The problem addressed in this project is twofold: first, how to accurately predict the Remaining Useful Life of aircraft engines from multivariate sensor data; and second, how to convert these predictions into an optimized, capacity-feasible maintenance schedule that minimizes cost while guaranteeing that no engine is ever operated beyond a safe limit. The challenge lies in developing a predictor that is both accurate and transparent, and an optimization model that is solved to verified, global optimality rather than approximated heuristically.

1.4 Proposed System

This project proposes a predictive maintenance framework for the CMAPSS FD001 turbofan fleet that couples an XGBoost-based Remaining Useful Life predictor with an Integer Linear Programming (ILP) optimization model. The system estimates the Remaining Useful Life of each engine and uses these estimates to generate a cost-optimal maintenance schedule that respects workshop capacity, budget, and resource limits, while never scheduling an engine past its safety deadline.

Figure 1.3 presents the overall architecture of the proposed system, illustrating the flow from sensor data acquisition to maintenance scheduling and dashboard delivery.

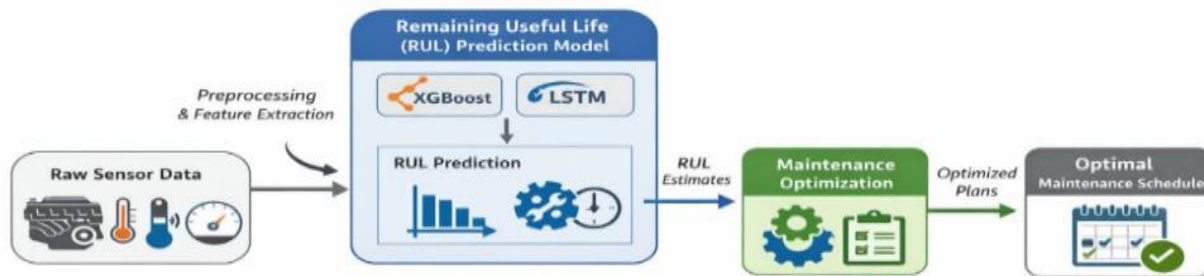


Figure 1.3: Overall framework of the proposed predictive maintenance system, from raw sensor data to an exported maintenance schedule

1.4.1 Aims and Objectives

- Develop and evaluate a data-driven XGBoost regression model to estimate the Remaining Useful Life of aircraft engines from the CMAPSS FD001 sensor data.
- Benchmark the predictor's accuracy against a published convolutional neural network (CNN) prognostic model evaluated on the same benchmark.
- Formulate the maintenance-scheduling task as an exact Integer Linear Program that consumes the predicted RUL directly and embeds an RUL-based safety-deadline constraint.
- Solve the optimization model to global optimality using the HiGHS solver, and verify its correctness against a hand-solved worked example and an independent spreadsheet check.
- Package the complete pipeline, from raw sensor data to an exported, ranked maintenance schedule, behind an interactive dashboard.

1.4.2 Proposed System Features

- Data preprocessing: Min-Max normalization of sensor readings and construction of a piecewise-linear RUL label capped at a healthy-phase ceiling.
- Feature engineering: raw sensor readings, rolling statistics, and degradation-delta features that measure each engine's drift from its own healthy baseline.
- XGBoost-based predictive model: an ensemble of gradient-boosted regression trees that estimates the Remaining Useful Life of each engine.
- Benchmarking: direct comparison of prediction accuracy against a published CNN prognostic model on the same dataset.
- Integer Linear Programming-based optimization: an exact scheduling model with a safety-deadline constraint, a budget constraint, a resource-availability constraint, and a workshop-capacity constraint, together with an emergency slot that guarantees the model always returns a feasible solution.
- Verification: a hand-solved worked example, cross-checked against an independent spreadsheet and the exact solver.
- Interactive dashboard: an end-to-end application connecting prediction and optimization, with exportable schedules and key performance indicators.

1.5 Development Methodology

The development of the predictive maintenance framework follows a structured approach to ensure accuracy, reliability, and efficiency. The methodology adopted for this project includes the following steps:

1. Data Collection acquire the NASA CMAPSS FD001 subset, comprising complete run-to-failure trajectories for one hundred training engines and truncated trajectories for one hundred test engines, together with the corresponding ground-truth RUL file.

2. Data Preprocessing rescale the retained sensor channels using Min-Max normalization and construct a piecewise-linear RUL label, held at a healthy-phase ceiling and decreasing linearly thereafter.
3. Exploratory Data Analysis (EDA) examine sensor trajectories, fleet-mean degradation behavior, inter-sensor correlation, and the distribution of engine lifetimes.
4. Model Development engineer raw, rolling-statistic, and degradation-delta features, then train an XGBoost regressor to estimate RUL; evaluate the model using RMSE and MAE, and benchmark it against a published CNN prognostic model evaluated on the same subset.
5. Maintenance Optimization formulate an Integer Linear Program that consumes the predicted RUL, embeds a safety-deadline constraint tied to the predictor's own measured error, and adds budget and resource-availability constraints; solve the model to global optimality with the HiGHS solver.
6. Verification hand-solve a small worked example, cross-check it against an independent spreadsheet calculation and against the solver's output, to confirm the implementation matches the model specification.
7. Deployment package the full pipeline behind an interactive dashboard that carries a user from raw data to an exported, ranked maintenance schedule.

1.6 Resource Requirements

The successful development and implementation of the predictive maintenance framework require the following resources:

1. Datasets: the NASA CMAPSS FD001 subset — 100 training engines (20,631 cycle records) with complete run-to-failure trajectories, 100 test engines (13,096 cycle records) truncated before failure, and a corresponding RUL ground-truth file.
2. Software Tools: Python, together with XGBoost and scikit-learn for model development and preprocessing; an Integer Linear Programming formulation solved with the open-source HiGHS solver; an interactive dashboard framework for the user interface; and a plotting library for charts and visualizations.
3. Hardware Requirements: a standard computer with sufficient memory to process the dataset. Because the predictive model is tree-based rather than a deep neural network, no GPU is required, and the model trains in a fraction of the time needed by the CNN benchmark.
4. Documentation Tools: Microsoft Word for report writing, and Microsoft Excel for the independent spreadsheet verification of the optimization model.

1.7 Report Layout

- 1. Introduction:** This section provides background information about the predictive maintenance framework for aircraft engines, including its purpose, objectives, and scope.
- 2. The Dataset:** This section describes the NASA CMAPSS FD001 subset, its structure, and the preprocessing steps applied to it, including normalization and RUL-label construction.
- 3. Exploratory Data Analysis:** This section presents an initial exploration of the data, including sensor trends, correlation structure, and the distribution of engine lifetimes, supported by visualizations.

4. Model Development: This section details the design, configuration, training, and evaluation of the XGBoost regression model used to predict Remaining Useful Life, together with its benchmarking against a published CNN prognostic model.

5. Maintenance Optimization: This section formulates the Integer Linear Programming model that converts predicted RUL values into an optimal maintenance schedule, and presents a hand-solved worked example that verifies the implementation.

6. System Implementation and Interface: This section describes the software architecture and the interactive dashboard that connects prediction and optimization.

7. Results and Discussion: This section presents the fleet-level results of the predictive model and the optimization schedule, and discusses their operational implications.

8. Beneficiary Organizations: This section identifies the organizations that stand to benefit from the proposed approach.

9. Conclusion and Future Work: This section summarizes the main findings, highlights the contributions of the project, and suggests directions for future work.

10. References: This section lists all sources cited in the report, following a consistent citation style.

Chapter 2

Background / Existing Work

2.1 Introduction

This chapter presents the theoretical foundations, design rationale, and existing research that support the proposed predictive maintenance framework. The focus of this work is on developing an applicable, decision-oriented system that combines a gradient-boosted regression model with an exact optimization model. In particular, this chapter explains how engineered, degradation-aware features are used to estimate Remaining Useful Life (RUL) with XGBoost, and how these predictions are translated into an optimal maintenance schedule through Integer Linear Programming (ILP). The chapter also positions the proposed approach within the existing literature and highlights the research gaps it addresses.

2.2 Overview of the Project

The objective of this project is to design an end-to-end predictive maintenance system for aircraft engines using multivariate sensor data collected over time. The system estimates the Remaining Useful Life (RUL) of each engine and uses these estimates to support optimal maintenance-scheduling decisions.

The proposed workflow consists of the following stages:

- Acquisition of raw multivariate time-series sensor data from the CMAPSS FD001 subset.
- Preprocessing of the sensor data, including Min-Max normalization and construction of a piecewise-linear RUL label.
- Feature engineering, combining raw sensor readings, rolling statistics, and degradation-delta features.
- RUL prediction using an XGBoost regression model.
- Benchmarking of the predictor's accuracy against a published convolutional neural network (CNN) prognostic model evaluated on the same subset.
- Optimization of maintenance actions using an Integer Linear Programming-based scheduling model that embeds a safety-deadline constraint derived from the predictor's own measured error.

A single predictive model, XGBoost, is adopted for RUL estimation. Its accuracy is evaluated against a convolutional neural network (CNN) prognostic model reported in the literature for the same benchmark, allowing the tree-based approach to be assessed for competitiveness against a deep-learning baseline while retaining the transparency and computational efficiency of a feature-based model.

2.3 Limitations of the Project

Despite the effectiveness of the proposed predictive maintenance framework, several limitations should be acknowledged.

First, the model is developed and evaluated using the FD001 subset of the CMAPSS dataset, which represents a single operating condition and a single fault mode under simulated conditions. Consequently, its performance may differ when applied to real-world aircraft engines, or to the harder, multi-condition CMAPSS subsets (FD002–FD004), which are not addressed in this project.

Second, the accuracy of the RUL predictions depends on the quality of the sensor data. Although the CMAPSS channels used here have already been filtered to remove constant or uninformative sensors, noise or sensor malfunction in a real deployment could still affect prediction quality.

Third, the scheduling model is deterministic: it treats each predicted RUL as a single value and absorbs prediction uncertainty through a fixed safety margin, equal to the predictor's measured root-mean-square error, rather than through an explicit probability distribution over possible outcome.

Additionally, the project focuses on a single predictive architecture, XGBoost, benchmarked against one published CNN baseline. Other architectures, such as Transformers or attention-based recurrent networks, are not explored in this project due to time and computational constraints.

Finally, the maintenance-optimization model relies on predefined cost, capacity, and resource parameters. These parameters, listed in Chapter 5, are drawn from the reference literature or set as explicit modelling and policy assumptions, and may not fully reflect the operational constraints of a specific airline or workshop.

2.4 Innovations in the Project

This project introduces several innovative aspects in the context of aircraft engine maintenance. Unlike traditional maintenance approaches that rely on fixed inspection intervals, the proposed system adopts a data-driven predictive maintenance strategy based on Remaining Useful Life (RUL) estimation, enabling maintenance decisions to be made proactively rather than reactively.

One of the key innovations of the project is the use of engineered, degradation-aware features — raw sensor readings, rolling statistics, and a degradation-delta measure of each engine's drift from its own healthy baseline to let a gradient-boosted regression model (XGBoost) match the accuracy of a published deep-learning benchmark while remaining far faster to train and fully transparent through feature-importance analysis.

Additionally, the project tightly integrates RUL prediction with maintenance scheduling: the predicted RUL feeds directly into an Integer Linear Programming model whose safety-deadline constraint is derived from the predictor's own measured error, making the safety margin statistically grounded rather than an arbitrary buffer.

Furthermore, the optimization model includes an emergency-slot mechanism that guarantees the model always returns a feasible solution: any engine that cannot be safely placed in a regular maintenance period is honestly routed to corrective maintenance rather than causing the model to fail. The implementation is verified end to end through a hand-solved worked example, cross-checked against both an independent spreadsheet calculation and the exact HiGHS solver.

Together, these innovations contribute to a coherent, verifiable, and practical predictive maintenance framework for aircraft engines, in which prediction and optimization reinforce rather than merely follow one another.

2.5 Design of the Project

The proposed project is designed as a single linear pipeline that carries the fleet from raw sensor data to an exported maintenance schedule. The overall architecture is illustrated in Figure 2.1: the CMAPSS FD001 dataset is loaded, an XGBoost model turns it into a predicted-RUL vector, an Integer Linear Program turns that vector into an optimal schedule, and the schedule and its indicators are presented and exported through an interactive dashboard.

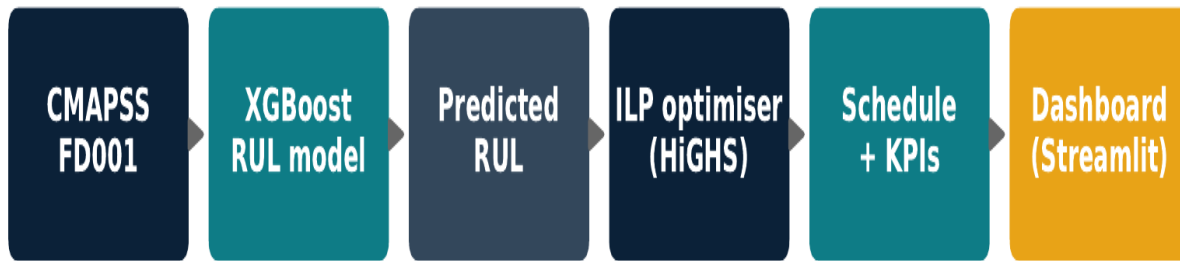


Figure 2.1: Proposed architecture from raw CMAPSS sensor data to an exported maintenance schedule

Technical Approach

Data Collection: The project utilizes the CMAPSS FD001 subset, which contains multivariate time-series sensor data collected from one hundred training engines (run to failure) and one hundred test engines (truncated before failure), across fourteen retained sensor channels.

Data Preprocessing: The collected data undergoes preprocessing to improve its quality and suitability for modelling. These steps include: rescaling all sensor channels to a common range via Min-Max normalization; constructing a piecewise-linear RUL label that is held at a healthy-phase ceiling and decreases linearly thereafter; and engineering raw, rolling-statistic, and degradation-delta features from the fourteen sensor channels.

Exploratory Data Analysis (EDA): Exploratory analysis is performed to understand sensor trends, inter-sensor correlation, and the distribution of engine lifetimes, guiding the feature-engineering and modelling choices described in Chapter 3.

Model Design and Training: An XGBoost regressor with a squared-error objective is trained on the engineered feature set to estimate the RUL of each engine, using a moderate tree depth, a small learning rate, and row/column subsampling for regularization.

Prediction and Maintenance Planning: The trained model generates a predicted RUL for each engine, which is then consumed directly by an Integer Linear Programming model to produce a cost-optimal, capacity-feasible, and safety-constrained maintenance schedule.

System Output: The final system outputs the predicted RUL for every engine, a ranked maintenance schedule, key performance indicators, and visual reports, all accessible through the interactive dashboard and exportable for further use.

2.6 Existing Work

This section reviews existing industrial systems and academic studies on predictive maintenance and Remaining Useful Life estimation for aircraft engines.

GE Aviation Engine Health Monitoring: General Electric has implemented predictive maintenance systems for aircraft engines, using sensor data to monitor engine performance continuously and to detect potential failures early, reducing operational disruptions.

Airbus Skywise Predictive Maintenance Platform: Airbus uses the Skywise platform to collect and analyze operational data from aircraft fleets, providing maintenance recommendations to airlines based on observed trends in engine performance.

Rolls-Royce TotalCare Program: Rolls-Royce's TotalCare program uses sensor data and predictive analytics to monitor engine components and plan maintenance activities, improving reliability and extending engine life.

Zheng et al. (2017): applied deep neural networks (DNN) for aircraft engine RUL estimation from time-series sensor data, focusing solely on prediction performance.

de Pater, Reijns and Mitici (2022): used a convolutional neural network (CNN) to predict the RUL of aircraft engines across the CMAPSS subsets and integrated Integer Linear Programming with a Genetic Algorithm for predictive maintenance scheduling and cost optimization. Their reported CNN accuracy is used in this project as the direct benchmark for the XGBoost predictor.

Dida et al. (2025): proposed an Attention-LSTM model for aircraft engine RUL prediction to capture long-term temporal dependencies, without integrating the results into maintenance scheduling.

2.7 Theoretical Background

2.7.1 Predictive Maintenance Concepts

Predictive maintenance is a data-driven strategy that anticipates failures by monitoring equipment condition over time. By identifying degradation trends early, maintenance actions can be scheduled proactively, reducing unplanned downtime and improving resource utilization.

2.7.2 Remaining Useful Life (RUL) Estimation

RUL is defined as the estimated remaining operational time before a system reaches a failure threshold. In this project, RUL estimation is formulated as a supervised regression problem, using a piecewise-linear label that is held at a healthy-phase ceiling for the early portion of an engine's life and decreases linearly thereafter. This avoids forcing the model to fit the long, uninformative early region during which sensors are essentially flat, and the predicted RUL serves as a quantitative indicator of maintenance urgency that directly drives scheduling decisions.

2.7.3 Feature Engineering for Time-Series Sensor Data

Because a tree-based regressor consumes fixed-length feature vectors rather than raw sequences, three groups of features are derived from the sensor channels at each cycle: the raw sensor readings, giving the instantaneous state of the engine; rolling statistics (mean and standard deviation over a short trailing window), which smooth measurement noise and expose the local trend; and a degradation-delta feature for each sensor, equal to the current reading minus that engine's own first reading, which measures accumulated wear independently of each engine's starting baseline.

2.7.4 Gradient Boosting and XGBoost

XGBoost is an implementation of gradient-boosted decision trees. The method builds an ensemble of shallow regression trees in sequence, where each new tree is fitted to the errors that the current ensemble still makes; the trees are then summed, with a small learning rate controlling how much each contributes. The result is a flexible, non-linear model that handles tabular inputs well, is robust to differently scaled features, and — unlike a deep network — exposes which features drive its predictions, a property that matters when a prediction is about to drive a costly, safety-relevant decision.

2.7.5 Benchmarking Against Deep-Learning Prognostics

To place the XGBoost model's accuracy in context, it is compared against a convolutional neural network (CNN) prognostic model reported in the literature for the same CMAPSS FD001 benchmark. Both models are evaluated using the same metric — root-mean-square error (RMSE), in cycles — so the comparison is direct and does not require re-implementing the deep-learning model.

2.7.6 Optimization and Maintenance Scheduling

Once a predicted RUL is available for every engine, an Integer Linear Programming (ILP) model converts these predictions into a maintenance schedule.

Decision Variables: a binary variable indicating whether an engine receives regular (preventive) maintenance in a given period, and a second binary variable indicating whether an engine is instead routed to an emergency (corrective) maintenance slot.

RUL-Driven Quantities: the predicted RUL is used to derive, for each engine, a preferred target maintenance period and a hard safety deadline, the latter obtained by subtracting a safety margin — equal to the predictor's own measured RMSE — from the predicted RUL.

Objective Function: minimize the total maintenance cost, combining an earliness/lateness service cost built around each engine's target period with a high penalty for any engine routed to the emergency slot.

Constraints: each engine is assigned to exactly one option, either a single regular period or the emergency slot; the workshop's per-period capacity is never exceeded; no engine receives regular maintenance after its RUL-based safety deadline; total expenditure remains within budget; and per-period resource availability (spare parts, crews) is respected.

Solution Method: because every decision variable is binary, the model is solved exactly by branch-and-bound, using the open-source HiGHS solver, returning a globally optimal — not approximate — schedule.

2.8 Related Models and Techniques Used in Literature

The literature reports a range of approaches for RUL estimation, spanning statistical models, classical machine learning, and deep-learning architectures. Deep neural networks and CNNs are effective at capturing degradation patterns directly from sensor sequences, attention-based recurrent networks capture long-term temporal dependencies, and gradient-boosting methods often provide strong performance on engineered, structured features. On the scheduling side, prior work has combined RUL prediction with Integer Linear Programming solved approximately via a Genetic Algorithm. This project instead solves the ILP exactly using the HiGHS solver, trading the flexibility of a metaheuristic for a certified, globally optimal schedule.

2.9 Summary of Existing Datasets

Benchmark datasets such as NASA's CMAPSS dataset are widely used for evaluating RUL prediction models. This project uses the FD001 subset specifically, which represents a single operating condition and a single fault mode, keeping the prognostic problem clean and letting the work concentrate on the prediction-to-scheduling pipeline. The harder, multi-condition subsets (FD002–FD004) are referenced only for benchmarking context and are not addressed by the models developed here.

2.10 Gap Analysis

Although extensive research exists on RUL prediction, limited work integrates a transparent, feature-based predictor with an exactly solved optimization model. Prior studies that combine deep-learning prognostics with maintenance scheduling typically rely on approximate metaheuristics, such as Genetic Algorithms, to solve the scheduling problem, and rarely tie the safety margin of the schedule to a statistically measured property of the predictor itself. This project addresses these gaps by proposing a framework that combines an XGBoost predictor benchmarked against a published CNN, an Integer Linear Program solved to global optimality, and a safety-deadline constraint derived directly from the predictor's own measured error.

2.11 Comparison with Previous Studies

Compared to previous studies, this project emphasizes both predictive competitiveness and exact, verifiable optimization. The XGBoost predictor is benchmarked directly against the CNN accuracy reported by de Pater, Reijns and Mitici (2022) on the same FD001 subset, while the scheduling problem, rather than being solved approximately with a Genetic Algorithm as in that reference study, is solved to certified global optimality with the HiGHS solver and independently verified through a hand-solved worked example and a spreadsheet cross-check.

2.12 Chapter Summary

This chapter presented the background, theoretical foundations, and existing research relevant to the proposed predictive maintenance framework. It justified the selection of XGBoost as the predictive model, explained the feature-engineering strategy used to represent time-series sensor data, described the role of CNN benchmarking, and detailed the Integer Linear Programming model used for maintenance scheduling. The next chapter describes the CMAPSS FD001 dataset and the exploratory data analysis that informs the modelling choices made in later chapters.

Chapter 3

Data Analysis

3.1. Data Overview

This section provides an overview of the dataset used in this project. The dataset is obtained from the **NASA Turbofan Engine Degradation Simulation dataset (FD001)**, which is widely used for predictive maintenance and Remaining Useful Life (RUL) estimation of aircraft engines.

The dataset consists of multivariate time-series data collected from multiple turbofan engines operating under normal conditions until failure. Each engine is monitored using several sensors that record operational and performance-related measurements over time.

The dataset includes:

- Multiple engine units, where each unit represents a single engine.
- Sequential operational cycles for each engine.
- Sensor measurements that reflect the health condition of the engine.
- Operating settings that describe different working conditions.

The data is divided into training and testing sets. The training set contains full engine run-to-failure records, while the testing set includes partial engine life data without the failure point. This structure enables the development and evaluation of predictive models for estimating the Remaining Useful Life of aircraft engines.

Here's a sample from the data

ID	Cycle	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21
1	1	518.67	641.82	1589.7	1400.6	14.62	21.61	554.36	2388.06	9046.19	1.3	47.47	521.66	2388.02	8138.62	8.4195	0.03	392	2388	100	39.06	23.419
1	2	518.67	642.15	1591.82	1403.14	14.62	21.61	553.75	2388.04	9044.07	1.3	47.49	522.28	2388.07	8131.49	8.4318	0.03	392	2388	100	39	23.4236
1	3	518.67	642.35	1587.99	1404.2	14.62	21.61	554.26	2388.08	9052.94	1.3	47.27	522.42	2388.03	8133.23	8.4178	0.03	390	2388	100	38.95	23.3442
1	4	518.67	642.35	1582.79	1401.87	14.62	21.61	554.45	2388.11	9049.48	1.3	47.13	522.86	2388.08	8133.83	8.3682	0.03	392	2388	100	38.88	23.3739
1	5	518.67	642.37	1582.85	1406.22	14.62	21.61	554	2388.06	9055.15	1.3	47.28	522.19	2388.04	8133.8	8.4294	0.03	393	2388	100	38.9	23.4044
1	6	518.67	642.1	1584.47	1398.37	14.62	21.61	554.67	2388.02	9049.68	1.3	47.16	521.68	2388.03	8132.85	8.4108	0.03	391	2388	100	38.98	23.3669
1	7	518.67	642.48	1592.32	1397.77	14.62	21.61	554.34	2388.02	9059.13	1.3	47.36	522.32	2388.03	8132.32	8.3974	0.03	392	2388	100	39.1	23.3774
1	8	518.67	642.56	1582.96	1400.97	14.62	21.61	553.85	2388	9040.8	1.3	47.24	522.47	2388.03	8131.07	8.4076	0.03	391	2388	100	38.97	23.3106
1	9	518.67	642.12	1590.98	1394.8	14.62	21.61	553.69	2388.05	9046.46	1.3	47.29	521.79	2388.05	8125.69	8.3728	0.03	392	2388	100	39.05	23.4066
1	10	518.67	641.71	1591.24	1400.46	14.62	21.61	553.59	2388.05	9051.7	1.3	47.03	521.79	2388.06	8129.38	8.4286	0.03	393	2388	100	38.95	23.4694
1	11	518.67	642.28	1581.75	1400.64	14.62	21.61	554.54	2388.05	9049.61	1.3	47.15	521.4	2388.01	8140.58	8.434	0.03	392	2388	100	38.94	23.4787
1	12	518.67	642.06	1583.41	1400.15	14.62	21.61	554.52	2388.09	9049.37	1.3	47.18	521.8	2388.02	8134.25	8.3938	0.03	391	2388	100	39.06	23.366
1	13	518.67	643.07	1582.19	1400.83	14.62	21.61	553.44	2388.12	9046.82	1.3	47.38	521.85	2388.08	8128.1	8.4152	0.03	393	2388	100	38.93	23.2757
1	14	518.67	642.35	1592.95	1399.16	14.62	21.61	554.48	2388.09	9047.37	1.3	47.44	521.67	2388	8134.43	8.3964	0.03	393	2388	100	39.18	23.3826
1	15	518.67	642.43	1583.82	1402.13	14.62	21.61	553.64	2388.11	9052.22	1.3	47.3	522.5	2388.08	8127.56	8.4199	0.03	391	2388	100	38.99	23.35
1	16	518.67	642.13	1587.98	1404.5	14.62	21.61	553.94	2388.05	9049.34	1.3	47.24	521.49	2388.07	8136.11	8.3936	0.03	392	2388	100	38.97	23.455
1	17	518.67	642.58	1584.96	1399.95	14.62	21.61	553.8	2388.06	9054.92	1.3	47.12	521.89	2388.04	8137.27	8.4542	0.03	392	2388	100	38.81	23.3319
1	18	518.67	642.62	1591.04	1396.12	14.62	21.61	554.2	2388.05	9049.55	1.3	47.21	521.76	2388.07	8132.73	8.4028	0.03	392	2388	100	38.89	23.3987
1	19	518.67	641.79	1587.56	1400.35	14.62	21.61	554.18	2388.04	9053.99	1.3	47.4	521.89	2388.03	8129.13	8.4321	0.03	391	2388	100	38.8	23.3464
1	20	518.67	643.04	1581.11	1405.23	14.62	21.61	554.81	2388.05	9045.9	1.3	47.22	522.07	2388.02	8129.71	8.421	0.03	392	2388	100	39.03	23.422
1	21	518.67	642.37	1586.07	1398.13	14.62	21.61	554.08	2388.11	9048.15	1.3	47.15	522.42	2388.08	8134.02	8.4049	0.03	392	2388	100	39.09	23.3101
1	22	518.67	642.77	1592.93	1400.57	14.62	21.61	553.63	2388.04	9061.21	1.3	47.24	522	2388.03	8130.41	8.4034	0.03	392	2388	100	38.92	23.3792
1	23	518.67	642.14	1588.19	1394.75	14.62	21.61	553.98	2388.05	9046.28	1.3	47.25	521.52	2388.05	8127.9	8.424	0.03	392	2388	100	38.94	23.4562
1	24	518.67	642.38	1590.83	1398.81	14.62	21.61	553.49	2388.12	9043.76	1.3	47.44	522.13	2388.03	8133.88	8.3891	0.03	392	2388	100	39	23.3696
1	25	518.67	642.77	1594.1	1399.39	14.62	21.61	554	2388.02	9054.16	1.3	47.36	522.56	2388.02	8136.61	8.3917	0.03	393	2388	100	38.95	23.4288
1	26	518.67	642.16	1589.08	1396.07	14.62	21.61	554.11	2388.07	9047.11	1.3	47.26	522.28	2388.06	8131.15	8.426	0.03	394	2388	100	38.86	23.4149

Figure 3.1: Sample from the data

3.2 Dataset Description

The dataset used in this project is divided into three main files:

- **Training set (train.txt):**
This file contains multivariate time-series data for multiple aircraft engines. Each engine operates from the beginning of its life until failure. Each record represents a single operational cycle and includes several sensor measurements that describe the engine's condition over time.
- **Test set (test.txt):**
This file contains similar time-series data for different engines; however, these engines do not reach failure within the dataset. The Remaining Useful Life (RUL) for these engines is not included in this file and is provided separately.
- **RUL file (RUL.txt):**
This file contains the true Remaining Useful Life values corresponding to the engines in the test set. Each value represents the number of cycles remaining before engine failure.

Each record in the training and test datasets includes the following attributes:

- **ID:** A unique identifier for each engine unit.
- **Cycle:** The operational cycle number of the engine.
- **Setting1, Setting2, Setting3:** Operational settings that represent different working conditions of the engine.
- **S1–S21:** Sensor readings that capture various physical parameters such as temperature, pressure, vibration, and other performance-related measurements.

3.3 Data Preprocessing

Before developing the predictive maintenance model, it was essential to preprocess the dataset to ensure its quality, consistency, and suitability for analysis. Data preprocessing plays a critical role in improving model performance and ensuring reliable Remaining Useful Life (RUL) predictions. This phase focused on preparing the NASA Turbofan Engine dataset for effective analysis and modeling.

The preprocessing process began with examining the raw dataset to understand its structure and characteristics. Since the dataset consists of multivariate time-series sensor data collected over multiple operational cycles, careful handling was required to maintain temporal relationships.

Data cleaning was the first major step. The dataset was inspected for missing values, duplicated records, and inconsistencies. Missing or constant sensor readings were identified and handled appropriately to avoid misleading the learning process. Duplicate records, if present, were removed to prevent bias in the analysis. In addition, irrelevant or non-informative features were excluded to reduce noise and improve computational efficiency.

Next, **data normalization** was applied to the sensor measurements. Since sensor values vary in scale and range, normalization ensured that all features contributed equally to the model training process. This step was particularly important for machine learning algorithms, as it helps accelerate convergence and improves prediction accuracy.

After cleaning and normalization, the **Remaining Useful Life (RUL)** for each engine in the training dataset was calculated. RUL was derived based on the maximum operational cycle of each engine, representing the number of cycles remaining before failure. This calculated RUL served as the target variable for the predictive models.

Finally, the dataset was prepared for time-series modeling by organizing the data into sequences suitable for learning temporal dependencies. This structured preprocessing ensured that the data was clean, consistent, and ready for subsequent exploratory data analysis and model development.

3.4 Data Integrity Check

Before performing any data cleaning or preprocessing operations, the raw dataset was carefully examined to ensure its reliability and consistency. Several integrity checks were applied to verify the quality of the data and identify potential issues that could affect model performance.

1. **Missing Values:**

The dataset was thoroughly scanned to detect missing or null values across all attributes. The analysis showed that no significant missing values were present in the main features, which indicates a high level of data completeness.

2. **Duplicate Records:**

The dataset was examined for duplicated rows that could introduce redundancy or bias during the model training process. Any duplicated records identified were removed to maintain data integrity.

3. **Statistical Summary:**

Descriptive statistics were generated for all numerical attributes to gain insights into their distribution, range, mean, and variability. This step helped in understanding the overall behavior of sensor readings and operational settings.

4. **Outlier Detection:**

Sensor measurements were analyzed to identify extreme or abnormal values that may result from sensor noise or measurement errors. The Interquartile Range (IQR) method was applied to detect and report outliers, allowing informed decisions on how to handle such values during preprocessing.

3.5 Data Cleaning:

After performing the data integrity checks, a data cleaning process was applied to prepare the dataset for modeling. The main objective of this step was to improve data quality, reduce noise, and ensure consistency between the training and testing datasets.

The raw dataset contains **26 columns**, including the engine identifier (**ID**), operational cycle number (**Cycle**), three operational settings (**Setting1**, **Setting2**, **Setting3**), and **21 sensor measurements (S1–S21)**. The training dataset consists of **20,631 records**, while the testing dataset contains **13,096 records**.

Initial inspection showed that the dataset does not contain missing values or duplicate records. However, basic statistical analysis revealed that some sensor readings include noticeable outliers, and several operational setting variables exhibit near-zero variance, which limits their usefulness for predictive modeling.

Based on these observations, the following cleaning steps were applied:

First, the operational setting variables (**Setting1, Setting2, and Setting3**) were removed, as they showed near-zero variance and did not contribute meaningful information to the prediction task.

Next, sensor features with very low variance (below **0.001**) were eliminated, as such features provide little discriminatory power. This step helped reduce noise and improve the focus of the model on informative sensor data.

To handle extreme values, sensor readings were clipped using **the Interquartile Range (IQR)** method. Values below the lower bound and above the upper bound were limited to reduce the influence of outliers while preserving the overall data distribution.

After that, all remaining sensor values were normalized using **Min-Max scaling** to ensure that features with different scales contributed equally during model training.

Finally, the same cleaned sensor features were applied to both the training and testing datasets to maintain consistency. The Remaining Useful Life (RUL) was calculated for the training data based on the maximum operational cycle of each engine, while RUL values for the test data were obtained from the provided external file and aligned with the corresponding engine IDs.

3.6 Data Exploration

Overview:

In the Data Exploration phase, we thoroughly examined our dataset to uncover meaningful insights and patterns.

Key Steps & Findings:

- **Descriptive Statistics:** Calculated mean, median, and standard deviation to understand central tendencies and variability.
- **Visual Analysis:** Used histograms, box plots, and scatter plots to inspect distributions, detect outliers, and examine relationships.
- **Correlation Exploration:** Analyzed correlations between sensors to identify significant dependencies and potential influential features.
- **Insights Gained:**
 - Highlighted trends in sensor readings over time
 - Identified low-variance and noisy sensors
 - Detected outliers and anomalies for preprocessing
 - Informed feature selection and model strategy

Impact:

This exploration provided a clear understanding of the dataset's structure and guided the preprocessing, sensor selection, and LSTM sequence preparation for predictive maintenance modeling.

3.7 Data Visualization:

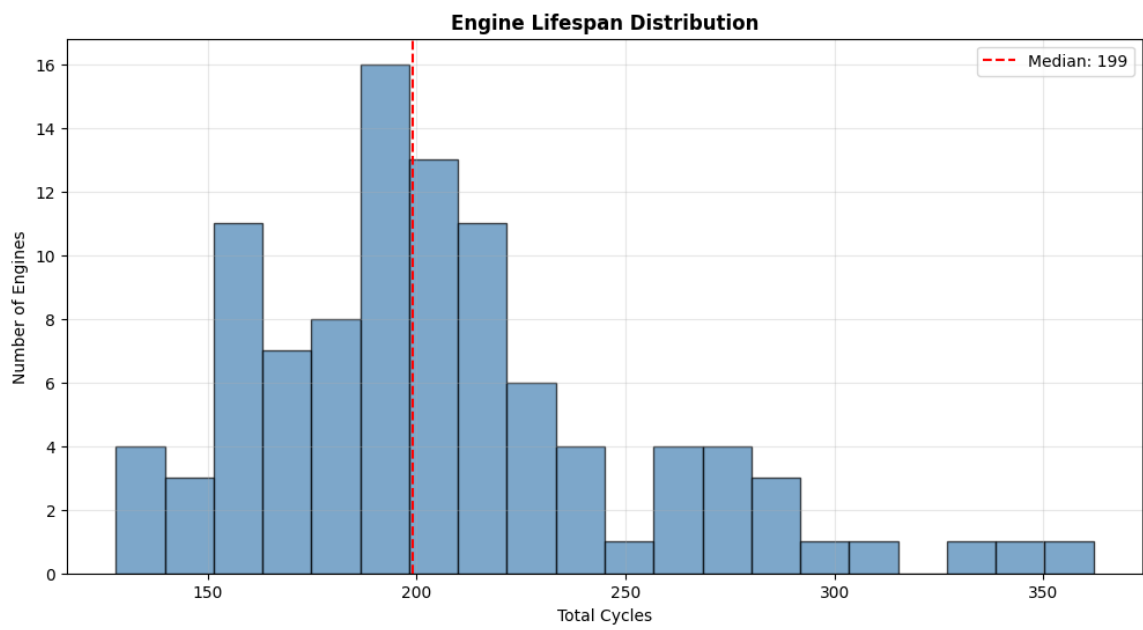


Figure 3.2: Engine Lifespan Distribution

Figure 3.2 displays the lifespan distribution of 100 turbfan engines from the FD001 dataset. The median lifespan equals 199 cycles, perfectly dividing engines into 49 engines with shorter lifespans below 199 cycles and 51 engines with longer lifespans above 199 cycles. This natural variability validates the RUL prediction framework and confirms realistic engine failure patterns.

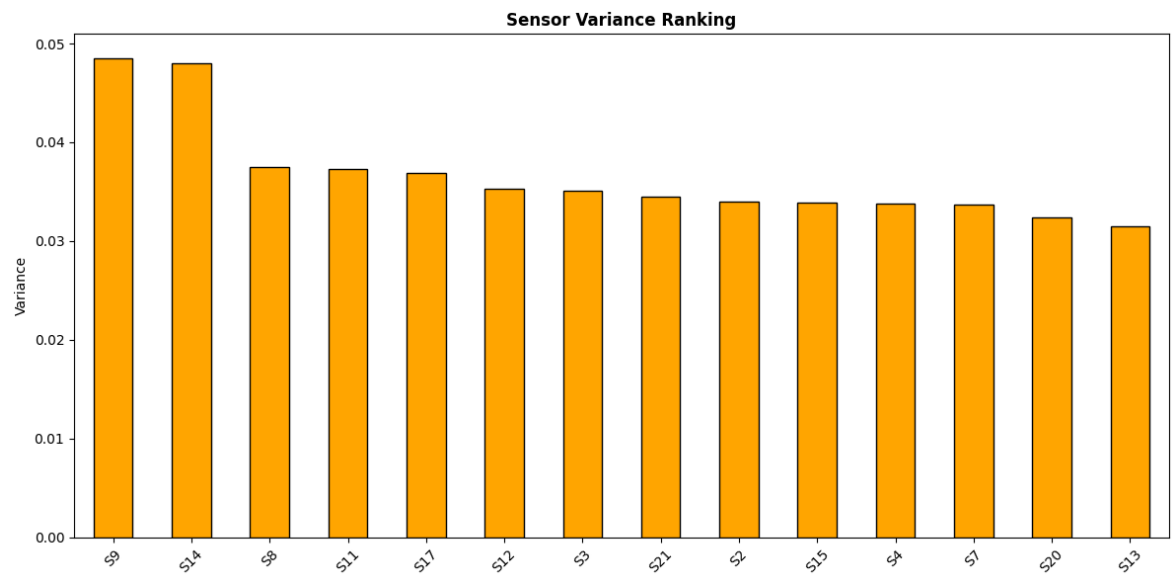


Figure 3.3: Sensor Variance Ranking

Figure 3.3 ranks the variance of 14 selected sensors after preprocessing. Higher variance indicates sensors that capture engine degradation patterns most effectively. Top-ranked sensors provide the majority of predictive information for RUL estimation while low-variance sensors were automatically removed during cleaning.

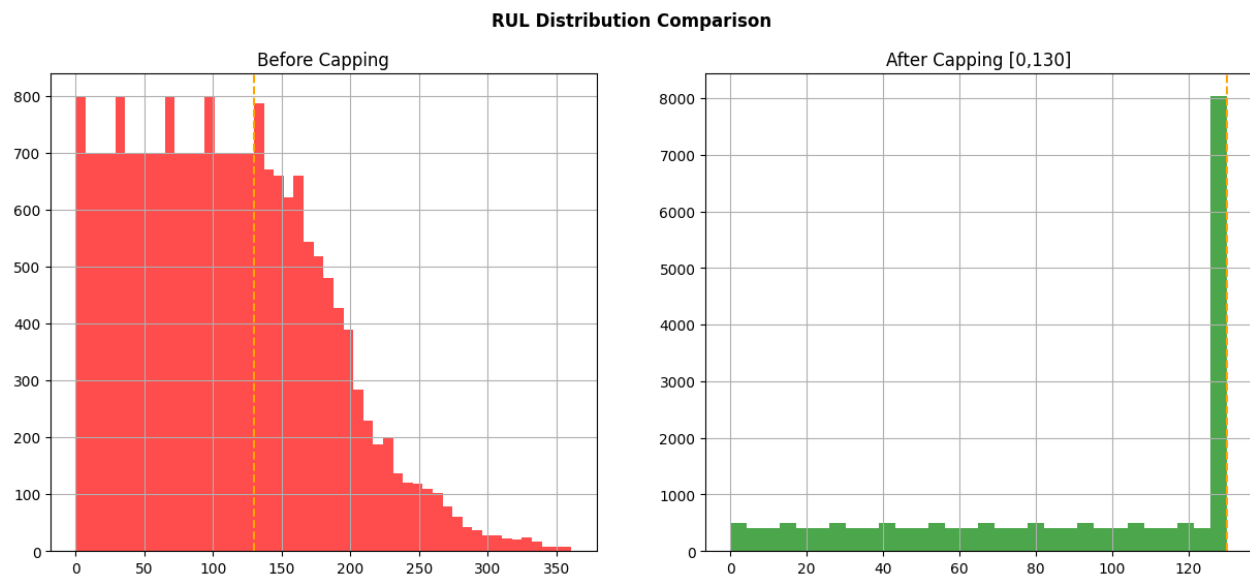


Figure 3.4: RUL Distribution Before/After NASA Capping

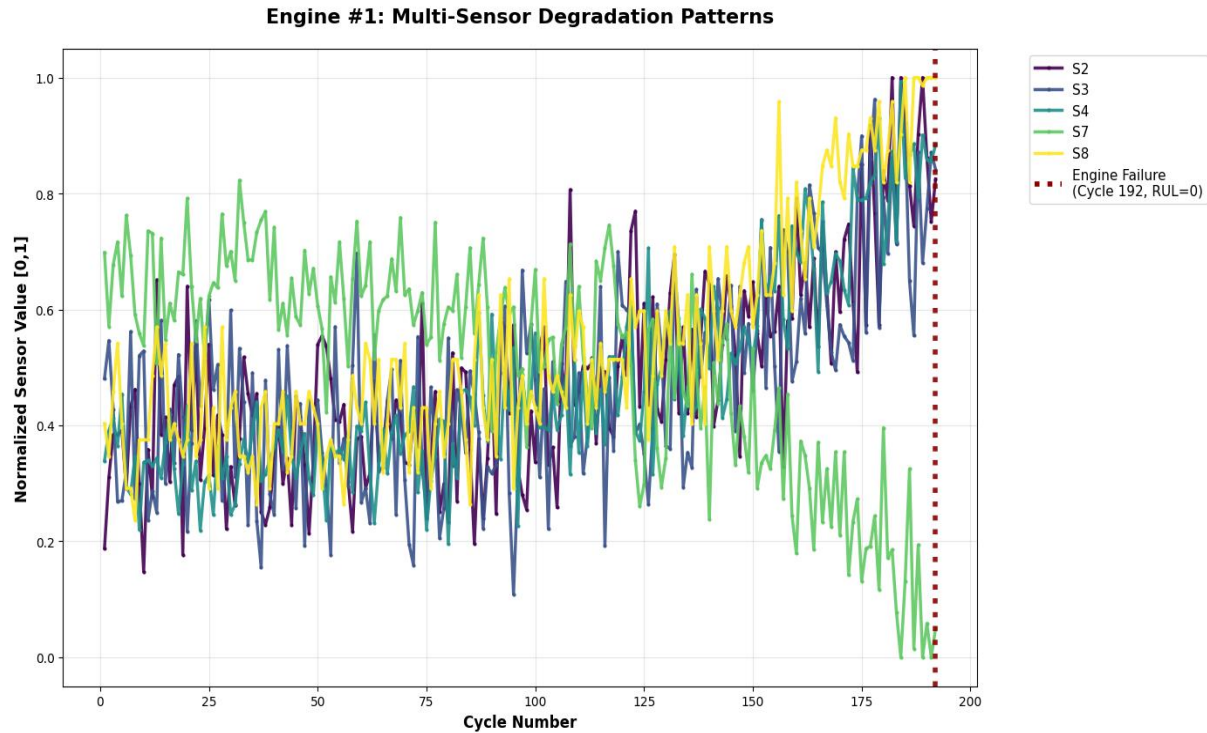
Figure 3.4 compares RUL distribution) before and after) NASA-standard 130-cycle capping across (20,631 observations). The left histogram shows raw RUL values where X-axis represents RUL values (0-362 cycles) and Y-axis (frequency) shows number of data points with each RUL value. Uncapped distribution displays unrealistic long tails where new engines show RUL=362 cycles - physically implausible since no maintenance engineer predicts exact failure 362 cycles ahead without degradation evidence.

Why exactly 130 cycles (not less, not more)?

NASA chose 130 as optimal balance between:

- Too low (<130 cycles): Compresses entire degradation phase into narrow range, reduces target variability limiting model learning, shortens maintenance warning window to impractical levels
- Too high (>130 cycles): Reintroduces unrealistic predictions during healthy phase, model learns artificial long-tail patterns instead of physical degradation signals
- 130 = Perfect compromise: ~70 cycles stable phase + 60-cycle safety margin = practical 4-month prediction horizon matching aviation maintenance cycles

Transformation Impact: Capping clips excessive healthy-phase RUL values while preserving all degradation information, creating realistic [0,130] target distribution where RUL=120 enables planned maintenance and RUL=20 triggers urgent action. This industry-standard preprocessing ensures models learn genuine physical degradation progression rather than mathematical extrapolation artifacts.



3.5:

#1 Multi-Sensor Degradation Trajectory

Figure
Engine

Figure 3.5 visualizes the complete operational lifecycle of (Engine #1 specifically) (one representative example from the FD001 dataset) through normalized readings from (five selected influential sensors) (**S2, S3, S4, S7, S8**) plotted against cycle numbers from the cleaned training dataset. These particular sensors were identified through feature importance analysis as the most predictive of degradation patterns among the 21 available sensor measurements. The chart reveals Engine #1's characteristic two-phase behavior where all five sensors maintain stable horizontal trajectories during initial cycles representing healthy operation with minimal wear.

A clear turning point becomes evident as these selected sensors simultaneously exhibit synchronized upward trends marking the onset of progressive deterioration that accelerates nonlinearly until reaching complete engine failure at the final cycle indicated by the dark red dotted line where RUL equals zero. The x-axis represents cycle progression from 1 to Engine #1's maximum observed cycles while the y-axis shows normalized sensor values scaled between 0 and 1 for comparability across different measurement units.

This specific visualization of Engine #1 demonstrates the piecewise degradation model fundamental to aviation prognostics where engines operate normally until crossing a critical physical threshold after which deterioration becomes both measurable and accelerating. The identical degradation patterns across these physically diverse yet highly-correlated sensors confirm strong interdependencies in the underlying failure mechanisms captured by NASA's FD001 dataset. This trajectory provides essential training signal for RUL prediction models by establishing both the stable baseline during healthy operation and the distinct degradation signature preceding failure across the entire engine fleet.

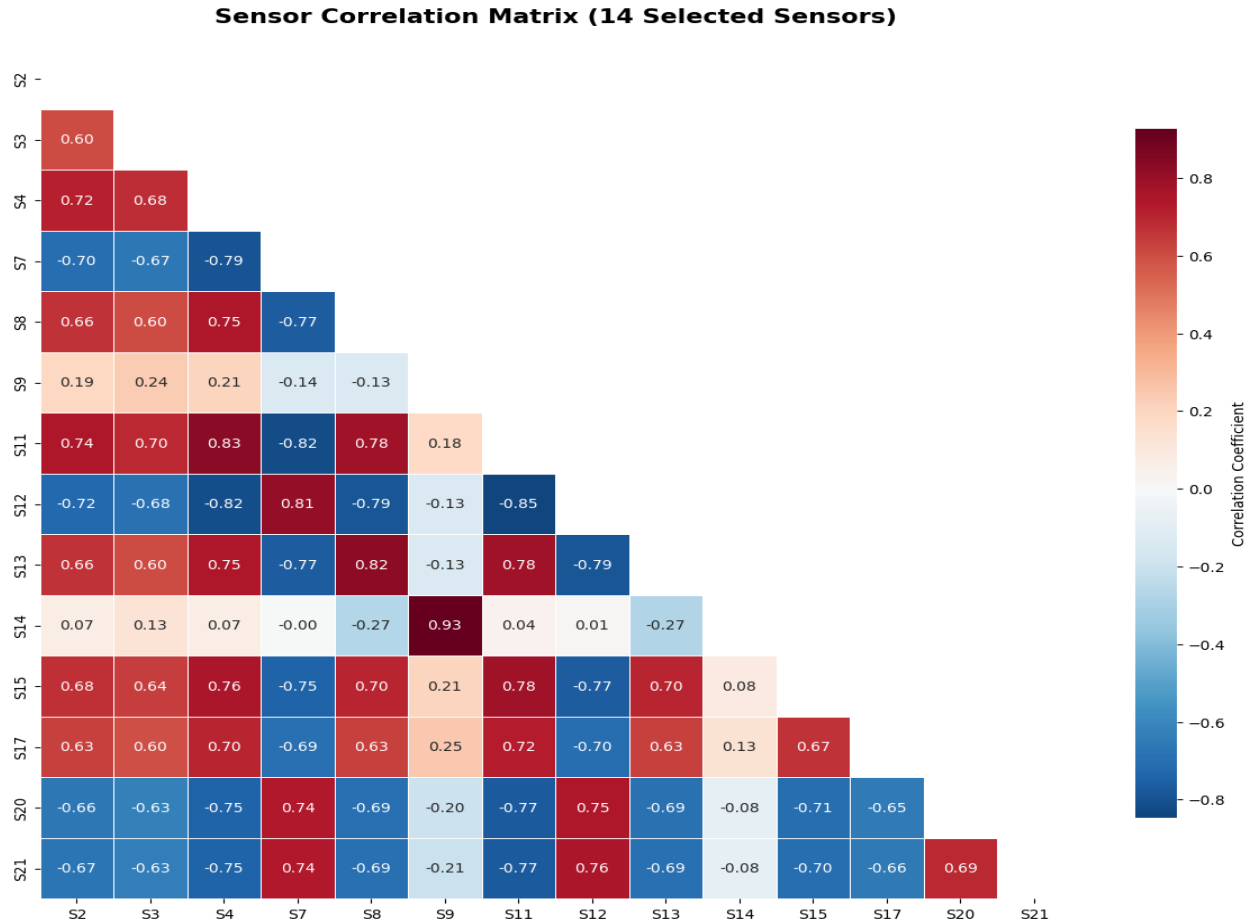


Figure 3.6: Sensor Correlation Matrix (Post-Cleaning Feature Selection)

Figure 3.6 displays the Pearson correlation matrix computed for the 14 highest-variance sensors identified through preliminary analysis from Figure 3.3. The lower triangular heatmap visualizes pairwise correlation coefficients ranging from -1.0 to +1.0, where values exceeding $|0.9|$ indicate highly collinear sensor pairs providing redundant degradation information. Notably, sensors **S9** and **S14** exhibited a correlation of **0.93**, triggering automated feature selection where **S14 was removed** as the lower-variance sensor according to Figure 3.3 variance rankings, preserving the higher-signal S9 while eliminating redundancy.

High correlation pairs were systematically identified through automated analysis, and from each redundant pair, the sensor exhibiting lower variance was selectively removed to eliminate multicollinearity while preserving maximum predictive signal strength. This variance-prioritized approach ensures retention of the most informative sensor from each physically-correlated group, systematically reducing feature dimensionality from 14 to 13 sensors without compromising degradation coverage. Strong positive correlations exceeding 0.9 confirm expected physical coupling in turbofan engine degradation processes, while moderate correlations between 0.6-0.9 across complementary sensor groups reveal multi-faceted failure signatures captured by the FD001 dataset. The resulting 13-sensor feature set eliminates redundancy while maintaining comprehensive prognostic capability, establishing an optimal balance between model complexity and predictive capacity. This systematic correlation-based pruning mitigates multicollinearity risks that could destabilize subsequent LSTM training while preserving all essential degradation patterns essential for accurate RUL prediction across operational engine fleets.

Chapter 4

Methodology

4.1 Introduction

This chapter presents the methodology developed to transform raw, multivariate sensor data into an executable and cost optimal maintenance decision for a fleet of turbofan engines. The methodology is organized around two connected stages that together form the core technical contribution of this project. In the first stage, the degradation of every engine is quantified through a supervised regression model that estimates its Remaining Useful Life from the sensor history recorded up to the last observed operating cycle. In the second stage, these estimates are consumed directly by an exact mathematical programming model that converts them into a maintenance schedule respecting workshop capacity, budget, and resource limits, while guaranteeing that no engine is ever operated beyond a safe point. A third, connecting activity benchmarks the accuracy of the predictive stage against a convolutional neural network reported in the literature for the same benchmark, so that the competitiveness of the chosen modelling approach can be established on objective grounds rather than asserted without evidence.

The purpose of this chapter is to explain, in a rigorous and reproducible manner, why each methodological choice was made and how the different stages of the pipeline connect to one another. Section 4.2 describes the methodology adopted for Remaining Useful Life prediction, including the formulation of the prediction task, the construction of the training target, the engineering of predictive features, the configuration of the regression model, and the metrics used to evaluate it. Section 4.3 describes the methodology used to benchmark the predictive model against a published deep learning baseline. Section 4.4 presents the optimization methodology, formulating the maintenance scheduling task as an Integer Linear Program and explaining the role of every decision variable, parameter, and constraint. The numerical results produced by applying this methodology, together with a hand solved worked example and a description of the software implementation, are presented separately in Chapter 5.

4.2 RUL Prediction Methodology

The predictive stage of this project is built around Extreme Gradient Boosting, commonly known as XGBoost, an ensemble learning method that has demonstrated strong performance on structured, tabular data across a wide range of application domains. This section describes how the Remaining Useful Life estimation task is formulated as a regression problem, how the training target is prepared, how predictive features are engineered from the raw sensor channels described in Chapter 3, how the model itself is configured, and how its performance is evaluated.

4.2.1 Problem Formulation as a Regression Task

Remaining Useful Life prediction is framed in this project as a supervised regression problem rather than as a classification or sequence modelling task. Under this formulation, every recorded operating cycle in the training fleet becomes an individual training example. The inputs to that example are the sensor derived features computed at that cycle, and the target is the number of operating cycles remaining until the corresponding engine reaches failure. Because the training set contains complete run to failure trajectories for every engine, as described in Chapter 3, the exact number of cycles remaining can be computed retrospectively for every single row in the training data, yielding a dense and fully labelled training set.

Once the regression model has been trained on this dense set of examples, it can be queried for any engine at any point in its operating life. In the context of this project, the model is queried specifically at the final observed cycle of each test engine, since this is the moment at which a maintenance planner would need an estimate of how much operating life remains.

The output of the model at that final cycle is taken as the fleet wide predicted Remaining Useful Life, which

becomes the single quantitative signal that drives every subsequent decision in the optimization stage described in Section 4.4.

4.2.2 Target Preparation

A naive, purely linear definition of the Remaining Useful Life target, computed simply as the number of cycles remaining until failure, creates a subtle but important difficulty for the learning process. Under such a definition, the target value keeps rising the further back in time one looks into an engine's history, even during the early portion of its life when the engine is healthy and its sensor readings are essentially flat. Forcing a regression model to fit this long and largely uninformative early region wastes model capacity and can reduce the accuracy of the predictions that matter most operationally, namely those close to the point of failure.

To address this difficulty, the training target adopted in this project follows the piecewise linear labelling scheme illustrated in Figure 3.2 of Chapter 3. Under this scheme, the Remaining Useful Life label is held constant at a ceiling of one hundred and twenty-five cycles throughout the healthy early portion of an engine's life, and only begins to decrease linearly once the engine has aged past that point. Predictions produced by the trained model are similarly clipped to the same ceiling, since distinguishing a very healthy engine from an extremely healthy engine carries no operational value for maintenance planning. This labelling strategy concentrates the learning signal on the degradation phase of an engine's life, which is precisely the phase in which an accurate estimate is required to support a safe and cost-effective maintenance decision.

4.2.3 Feature Engineering

Because XGBoost consumes fixed length feature vectors rather than raw sequential data, the fourteen sensor channels retained after the preprocessing steps described in Chapter 3 must first be converted into a set of engineered features at every operating cycle. Three complementary groups of features are constructed for this purpose.

The first group consists of the raw sensor readings at the current cycle, which provide the model with the instantaneous state of the engine. The second group consists of rolling statistics, specifically the mean and the standard deviation of each sensor computed over a short trailing window of cycles, which smooth out measurement noise and expose the local trend in the sensor signal. The third group consists of a degradation delta feature computed for every sensor, defined as the difference between the current reading and that same engine's own first recorded reading. This last feature provides an explicit, per engine measure of how far a unit has drifted from its own healthy baseline, independently of the absolute starting value of that particular engine.

Among these three groups, the degradation delta features are found to carry the largest share of predictive power, as confirmed by a grouped permutation importance analysis illustrated in Figure 4.1. This finding is intuitive once the underlying mechanism is considered. A model that only observes raw sensor readings cannot easily determine how far an engine has aged, because different engines begin their operating life at slightly different baseline readings. By subtracting each engine's own initial reading from its subsequent readings, the degradation delta feature turns an ambiguous absolute measurement into a clean, comparable measure of accumulated wear that is directly informative of remaining life.

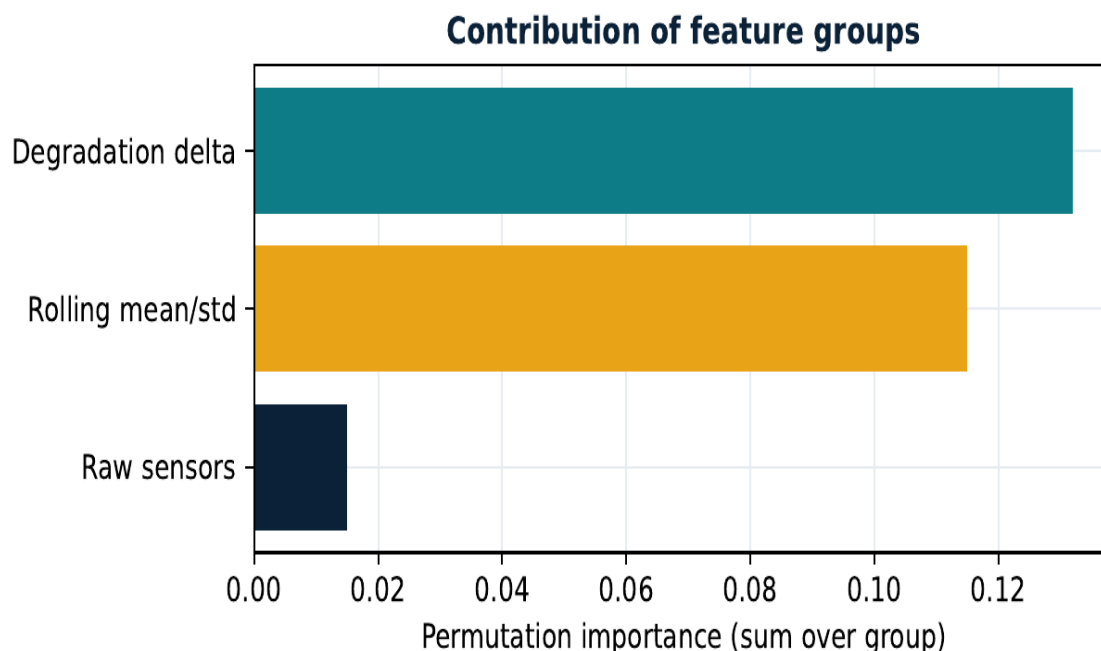


Figure 4.1: Contribution of the three engineered feature groups to model accuracy, measured by grouped permutation importance. The degradation delta group dominates the other two groups.

4.2.4 Model Configuration

XGBoost was selected as the predictive model for this project for several complementary reasons. As an implementation of gradient boosted decision trees, it builds an ensemble of shallow regression trees in sequence, where every new tree is fitted specifically to the residual errors that the current ensemble still makes, and the resulting trees are summed together with a small learning rate controlling the contribution of each one. This procedure produces a flexible, nonlinear model that handles tabular inputs naturally, remains robust to differently scaled input features, and, unlike a deep neural network, exposes directly which features drive its predictions through a feature importance analysis such as the one presented in Figure 4.1. These properties, together with a strong track record on structured, tabular prediction problems, make XGBoost particularly well suited to a task in which both accuracy and interpretability are required.

The predictor used in this project is an XGBoost regressor trained with a squared error objective function. The main configuration settings adopted for the model are summarized in Table 4.1. A moderate tree depth combined with a small learning rate and a large number of boosting rounds produces a smooth, well regularized fit, while row subsampling and column subsampling act as additional regularization mechanisms that reduce the risk of overfitting to the training fleet.

Setting	Value	Role
Boosting rounds	600	Number of trees included in the ensemble
Learning rate	0.03	Shrinkage applied to the contribution of each tree
Maximum depth	6	Controls the complexity of each individual tree
Subsample	0.8	Fraction of training rows used to build each tree
Column subsample	0.8	Fraction of features considered when building each tree
L2 regularization	1.0	Penalty applied to the leaf weights of each tree
Minimum child weight	30	Minimum amount of evidence required to justify a split

Table 4.1: Main configuration settings adopted for the XGBoost regression model.

4.2.5 Evaluation Metrics

Three complementary metrics are used to evaluate the accuracy of the trained model on the held-out test fleet. The root means square error, expressed in operating cycles, is treated as the primary measure of accuracy because it penalizes large individual errors more heavily than small ones, which is a desirable property in a safety relevant prediction task. The mean absolute error is reported alongside it because it provides an easily interpreted average miss, expressed directly in cycles, that is less sensitive to a small number of outlying predictions.

In addition to these two standard regression metrics, an asymmetric prognostic score commonly reported in the Remaining Useful Life literature is also considered. This score penalizes late predictions, meaning predictions that overestimate the true remaining life of an engine, more heavily than early predictions of the same magnitude. This asymmetry reflects the operational reality that an overly optimistic prediction is considerably more dangerous than an overly conservative one, since it risks allowing an engine to continue operating beyond the point at which it is actually safe to do so.

4.3 Benchmarking Methodology

To situate the accuracy of the XGBoost predictor within the broader body of published research, its performance is compared against a convolutional neural network reported in the literature as a prognostic model for the same benchmark. This benchmarking exercise is included in the methodology of this project because a predictive model that has not been compared against an established point of reference offers little evidence of its own competitiveness.

4.3.1 Reference Model

The reference model used for benchmarking purposes is the convolutional neural network reported by de Pater, Reijns, and Mitici in their 2022 study, which applied a convolutional architecture to the CMAPSS family of datasets as the prognostic front end of a maintenance scheduling study addressing essentially the same overall problem tackled in this project. The root means square error values reported by that study are 14.35, 17.98, 12.05, and 15.15 cycles on the FD001, FD002, FD003, and FD004 subsets of CMAPSS respectively.

4.3.2 Basis for Comparison

Both the reference convolutional neural network and the XGBoost model developed in this project are evaluated using the same metric, namely root mean square error expressed in cycles, which allows a direct and meaningful comparison to be drawn on the FD001 subset used throughout this project. The figures reported for the remaining, more difficult multi condition subsets of CMAPSS, namely FD002 through FD004, are reproduced only to provide context for the reader. Because the scope of this project is deliberately confined to the FD001 subset, as explained in Chapter 2, no claim of competitiveness is made regarding the performance of the XGBoost approach on those harder subsets.

4.4 Optimization Methodology

Once a predicted Remaining Useful Life value is available for every engine in the fleet, the remaining methodological task is to decide when each engine should be serviced. A simple approach of sorting the fleet by predicted Remaining Useful Life and servicing engines in that order does not constitute optimization, because it ignores the reality that any maintenance workshop can accommodate only a limited number of engines within a given period, and because it assigns no explicit cost to servicing an engine too early or too late relative to its ideal maintenance point. The genuine tradeoff between these competing considerations only becomes visible once a capacity limit is introduced, and it is precisely this trade off that the Integer Linear Program described in this section is designed to resolve.

4.4.1 Rationale

An Integer Linear Program was selected for the scheduling task because it allows the maintenance decision to be expressed as an explicit optimization problem with a verifiable, globally optimal solution, rather than as a heuristic ranking procedure whose quality cannot be certified. Every decision in the resulting model is represented by a binary variable, every operational limit is represented by an explicit linear constraint, and every cost consideration is represented by an explicit linear term in the objective function. This structure allows the resulting schedule to be inspected, defended, and reproduced by any party with access to the model and its solver, a property that matters considerably in a domain where maintenance decisions carry both financial and safety consequences.

4.4.2 Decision Variables

Two families of binary decision variables are used to describe a candidate maintenance schedule. The first variable, denoted x for engine i in period t , equals one when engine i is assigned to regular, preventive maintenance during period t , and equals zero otherwise. The second variable, denoted y for engine i , equals one when engine i is instead routed to an emergency, corrective maintenance slot, a fallback option that represents servicing the engine outside the normal preventive plan at a considerably higher cost. Every engine in the fleet is required to take exactly one of these two options, as formalized in the assignment constraint presented in Section 4.4.5.

4.4.3 RUL Driven Quantities

The predicted Remaining Useful Life of every engine is used to derive two operationally meaningful quantities, namely a preferred target maintenance period and a hard safety deadline, together with a service cost structure built around them.

The target period represents the maintenance period that would be preferred for a given engine, placed slightly ahead of its predicted end of life through a safety factor denoted by the Greek letter β , whose value lies between zero and one. The deadline represents the very last period during which regular, preventive maintenance can still safely be performed on that engine. It is derived by first subtracting a fixed safety margin, denoted by the Greek letter δ , from the predicted Remaining Useful Life before converting the result into a period index. Critically, this safety margin is set equal to the measured root mean square error of the XGBoost predictor itself, described in Section 4.2.5, so that the margin protecting the schedule is not an arbitrary buffer but a statistically grounded quantity tied directly to the predictor's own demonstrated accuracy.

$$t_{\text{tgt sub } i} = \text{ceil}(\beta \times \text{RUL}_i / \rho)$$

$$d_i = \text{floor}((\text{RUL}_i - \Delta) / \rho)$$

Here, the symbol rho represents the number of operating cycles consumed per planning period, allowing the model to translate a continuous prediction expressed in cycles into a discrete period index suitable for a scheduling decision. Once the target period has been established for every engine, a service cost is pre computed as a fixed parameter of the model, built around an earliness and lateness cost structure.

$$C_{i,t} = C_m + p_e \times [\text{target} - t]^+ + p_l \times [t - \text{target}]^+$$

In this expression, the bracketed terms with a superscript plus sign denote the positive part of the enclosed quantity, meaning that only genuinely early or genuinely late assignments contribute a penalty. Servicing an engine before its target period incurs a comparatively mild earliness penalty, reflecting the wasted operating life that results from bringing an engine in too soon, whereas servicing an engine after its target period incurs a considerably larger lateness penalty, reflecting the rising safety risk associated with delaying maintenance. Pre computing this cost as a fixed parameter, rather than expressing it as a nonlinear function of the decision variables, is what allows the overall scheduling model to remain a linear program.

4.4.4 Objective Function

The optimization model minimizes the total expected maintenance cost across the entire fleet, combining the regular service cost of every engine assigned to a preventive maintenance period with the emergency penalty of every engine routed to the corrective maintenance slot.

$$\min Z = \sum_{i,t} C_{i,t} x_{i,t} + \sum_i p_g y_i$$

Minimizing this single objective simultaneously encourages the model to service every engine as close as possible to its own target period, while discouraging the model from relying on the considerably more expensive emergency slot except where doing so is unavoidable given the capacity and safety constraints described below.

4.4.5 Base Constraints

Two constraints, adapted from the maintenance scheduling formulation of de Pater, Reijns, and Mitici, define what constitutes a valid schedule regardless of cost. The assignment constraint requires that every engine be assigned to exactly one option, either a single regular maintenance period or the emergency slot, ensuring that no engine is left unplanned and that no engine is serviced twice. The capacity constraint limits the total number of engines that can be assigned to regular maintenance within any single period to the workshop's known throughput.

It is this capacity constraint that transforms the scheduling task into a genuine optimization problem. When several urgent engines compete for the same early periods, the model must decide which of them to bring forward and which to defer to a slightly later period, a decision that can only be made correctly by weighing the relative cost of each option against the others, exactly the kind of trade off that an exact optimization model is designed to resolve.

4.4.6 Core Constraints of This Work

Beyond the two base constraints inherited from the reference literature, three further constraints form the specific methodological contribution of this project.

The first of these three constraints is the Remaining Useful Life based safety deadline, which forbids any engine from receiving regular maintenance after the deadline period derived for it in Section 4.4.3. This constraint exists because operating an engine beyond its predicted Remaining Useful Life, less the predictor's own measured error, risks an in-service failure. Tying the safety margin to the predictor's own root mean square error, rather than to an arbitrary fixed buffer, makes this constraint statistically grounded rather than an arbitrary design choice.

The second constraint is a budget constraint, which requires that the total expenditure implied by the schedule, combining regular service costs and emergency penalties, must not exceed an available maintenance budget. This constraint reflects the standard managerial reality that maintenance planning operates under a finite budget, a limit commonly encountered in preventive maintenance planning literature.

The third constraint is a resource availability constraint, which requires that the consumption of each limited maintenance resource, such as specialized crews or spare parts, within any given period must not exceed the amount of that resource available during that period. This constraint reflects the fact that maintenance activity consumes limited physical resources in addition to workshop time, a structure inherited from the resource constrained formulation commonly used in aircraft line maintenance scheduling research.

Constraint	Bounds	Reason for inclusion
Safety deadline	Time	Prevents an engine from operating beyond a statistically grounded safe limit
Budget	Cost	Reflects a finite, managerially imposed maintenance budget
Resource availability	Physical resources	Reflects limited crews and spare parts available per period

Table 4.2: The three core constraints introduced in this work, and the dimension of the scheduling problem that each one bounds.

Because these three constraints act upon different dimensions of the problem, namely time, cost, and physical resources respectively, they cannot contradict one another. The only conditions that must be verified externally are that the available budget is at least as large as the cost of the cheapest feasible assignment for the fleet, and that the available resources are sufficient across the planning horizon. Under these mild conditions, the resulting model is guaranteed to remain both feasible and internally consistent.

4.4.7 The Emergency Slot

The emergency decision variable introduced in Section 4.4.2 plays a specific structural role within the model. Because the safety deadline constraint forbids regular maintenance after an engine's deadline period, an engine whose deadline falls very early, or whose deadline period happens to already be full to capacity, might otherwise have no feasible regular period available to it, which would render the entire model infeasible. The emergency slot prevents this outcome from occurring. It remains available at all times and is left unrestricted by the capacity, deadline, and resource constraints described above, so that any engine that cannot safely be placed into a regular period is instead routed to corrective maintenance at a high but finite penalty.

This design decision reflects both an operational and a mathematical consideration. Operationally, an engine that cannot be safely scheduled within the available capacity genuinely is a corrective case, and the model is designed to report this outcome honestly rather than to conceal it. Mathematically, the presence of an always available emergency option guarantees that the assignment constraint introduced in Section 4.4.5 can always be satisfied, which in turn guarantees that the overall optimization model always returns a feasible solution regardless of how tightly constrained the fleet happens to be in a given instance.

4.4.8 Solution Method

Because every decision variable in the model is binary, the resulting formulation is an Integer Linear Program. Integer Linear Programs of this kind are solved to certified global optimality using a branch and bound procedure, which explores a tree of partial assignments while relying on linear relaxations of the original problem to prune branches that provably cannot improve upon the best solution found so far. Upon completion, this procedure returns a schedule that is provably optimal for the given input data and parameters, rather than a heuristic approximation whose quality cannot be guaranteed.

The open-source HiGHS solver is used throughout this project to carry out this branch and bound search. For the complete, one hundred engine instances addressed in this project, the solver returns a certified globally optimal schedule in a fraction of a second, a computational cost that is negligible compared with the time required to train the predictive model described in Section 4.2.

4.4.9 Parameter Values and Sources

Every parameter appearing in the optimization model requires a numerical value before the model can be solved. Table 4.3 lists each parameter together with the value adopted in this project and the source or justification for that value. Only two of these quantities represent genuine operational decisions that would need to be revisited for a different airline or workshop, namely the workshop capacity, which depends on the physical throughput of the facility in question, and the safety factor beta, which represents a policy choice about how far ahead of an engine's predicted end of life a maintenance intervention should be scheduled. The remaining parameters are either drawn directly from the reference literature, derived directly from the predictor's own measured accuracy, or adopted as explicit, clearly stated modelling assumptions.

Parameter	Value	Source
Cycles per period (ρ)	1	Modelling assumption
Safety factor (β)	0.90	Policy decision regarding maintenance timing
Safety margin (δ)	14	Equal to the predictor's measured RMSE
Workshop capacity	3	Operational decision reflecting workshop throughput
Preventive maintenance cost	10,000	Adopted from the reference study
Earliness penalty	100	Earliness weight adopted for this study
Lateness penalty	1,000	Adopted from the reference study
Emergency penalty	50,000	Approximately five times the preventive cost, following reliability practice

Table 4.3: Complete list of optimization model parameters, their adopted values, and the provenance of each value.

4.5 Chapter Summary

This chapter has presented the complete methodology underlying the predictive maintenance system developed in this project. The Remaining Useful Life of every engine is estimated through an XGBoost regression model trained on a carefully constructed, piecewise linear target and on a set of engineered features that capture both the instantaneous state and the accumulated degradation of each engine. The accuracy of this predictor is benchmarked directly against a convolutional neural network reported in the literature for the same dataset, establishing its competitiveness on objective grounds. The predicted Remaining Useful Life values are then consumed by an Integer Linear Program that produces a cost optimal, capacity feasible, and safety constrained maintenance schedule, solved to certified global optimality using the HiGHS solver.

Having established why each methodological choice was made and how the predictive and optimization stages connect to one another, the following chapter turns to the concrete results produced by applying this methodology. Chapter 5 presents a hand solved worked example that verifies the correctness of the optimization model, describes the software architecture and interactive dashboard used to implement the complete pipeline, and reports the fleet level prediction accuracy, optimal schedule, and key performance indicators obtained when the methodology described here is applied to the full one hundred engine fleet.

Chapter 5

Work Details

5.1 Introduction

The previous chapter explained why the predictive maintenance system was designed the way it was, covering the reasoning behind the choice of XGBoost as the prediction model, the construction of the training target, the engineering of predictive features, and the formulation of the Integer Linear Program used to schedule maintenance. This chapter turns from reasoning to evidence. Its purpose is to show, in concrete and verifiable terms, that the methodology described earlier actually works when it is applied to real data and solved with real software.

The chapter is organized into three broad parts. The first part walks through a small, hand solved worked example that traces every calculation performed by the optimization model on a handful of engines, so that the logic of the scheduling process can be followed without the distraction of a hundred engine fleet. This worked example is then checked twice, once against the output of the HiGHS solver and once again through an independent spreadsheet built in Microsoft Excel, so that three separate methods of computation agree on the same answer before any confidence is placed in the full-scale results. The second part describes how the complete system was implemented in software, covering the overall architecture of the pipeline, the technologies used to build it, the way a user interacts with the application, the dashboard that presents the results, and the export features that allow a maintenance engineer to take the output away and use it. The third part reports the actual results obtained when the methodology of Chapter 4 is applied to the full fleet of one hundred engines drawn from the CMAPSS FD001 benchmark, covering the prediction accuracy of the XGBoost model, its benchmarking against a convolutional neural network reported in the literature, the optimal maintenance schedule produced by the solver, the priority ranking of the fleet, the key performance indicators of the resulting plan, and a broader visual analysis that draws all of these threads together.

Throughout this chapter, tables and figures are used generously, since a scheduling and prediction system of this kind is considerably easier to evaluate visually than through prose alone. Wherever a screenshot or chart could not be reproduced directly from the available project material, a clearly marked placeholder has been inserted in its place, so that the structure of the final report is complete and ready to receive the corresponding image once it is available.

5.2 Worked Example

Before applying the optimization model to the full one hundred engine fleet, it is useful to work through a small instance by hand. A worked example of this kind serves two purposes. First, it demonstrates that the equations introduced in Chapter 4, namely the derivation of the target period, the derivation of the safety deadline, and the construction of the service cost, behave exactly as intended once real numbers are substituted into them. Second, it provides a transparent, independently checkable reference point against which the correctness of the software implementation can later be confirmed, since any discrepancy between a hand calculation and a solver result on a small instance would point immediately to an error in either the formulation or its implementation.

5.2.1 Instance Definition

The instance chosen for this worked example consists of five engines whose predicted Remaining Useful Life values are eight, fifteen, twenty two, thirty, and forty five cycles respectively. These five values were deliberately chosen to span a wide range of urgency, from an engine that is very close to the end of its predicted life to one that still has a considerable amount of operating life remaining, so that the resulting schedule illustrates how the model treats engines of very different priority within the same instance.

The parameters governing this instance are set deliberately small so that every intermediate quantity can be verified by hand. The number of operating cycles consumed per planning period is set to one, the safety factor beta is set to zero point nine, and the safety margin delta is set to five cycles. The workshop capacity is deliberately set to a single engine per period, which is considerably tighter than the capacity used in the full fleet study reported later in this chapter, and is chosen specifically so that the capacity constraint becomes active and forces the model to make a genuine scheduling decision rather than trivially accommodating every engine at once. The base preventive maintenance cost is set to ten, the earliness penalty per period is set to one, the lateness penalty per period is set to one hundred, and the emergency penalty for any engine routed to corrective maintenance is set to five hundred. Table 5.1 summarizes this complete set of inputs.

Parameter	Symbol	Value
Cycles per period	ρ	1
Safety factor	β	0.9
Safety margin	δ	5
Workshop capacity per period	$C_{sub\ t}$	1
Base preventive maintenance cost	$c_{sub\ m}$	10
Earliness penalty per period	$p_{sub\ e}$	1
Lateness penalty per period	$p_{sub\ l}$	100
Emergency penalty	$p_{sub\ g}$	500

Table 5.1: Parameters and predicted Remaining Useful Life values adopted for the five engine worked example.

The predicted Remaining Useful Life values for the five engines in this instance are listed again in Table 5.2, together with the engine identifiers used consistently throughout the remainder of this section.

Engine	Predicted RUL (cycles)
Engine 1	8
Engine 2	15
Engine 3	22
Engine 4	30
Engine 5	45

Table 5.2: Predicted Remaining Useful Life for the five engines used in the worked example.

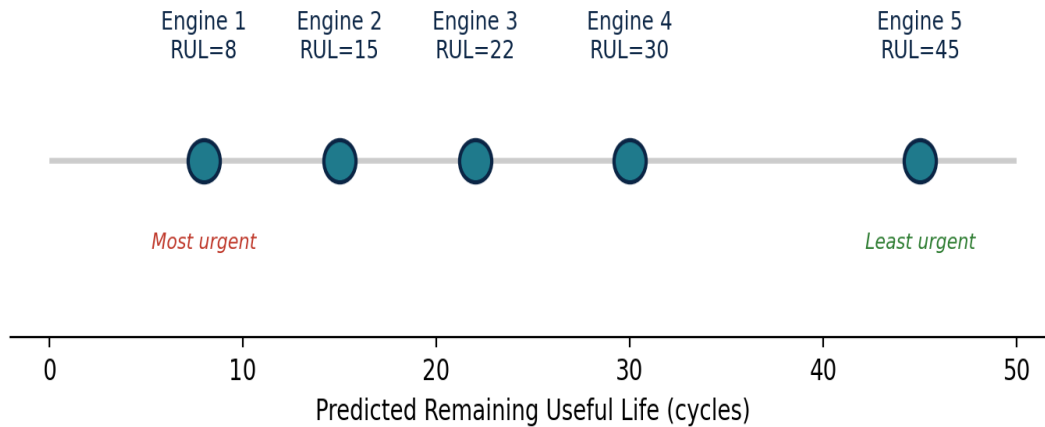


Figure 5.1: Illustrative timeline of the five engines in the worked example, ordered by predicted Remaining Useful Life.

5.2.2 Step by Step Solution (Targets, Deadlines, Cost)

The first calculation performed for every engine is the derivation of its target maintenance period, which represents the period in which the engine would ideally be serviced. The target period is computed by multiplying the predicted Remaining Useful Life by the safety factor beta and rounding the result upward to the nearest whole period, dividing by rho beforehand where rho differs from one.

$$t_{\text{tgt sub } i} = \text{ceil}(\beta \times \text{RUL}_i / \rho)$$

The second calculation performed for every engine is the derivation of its safety deadline, which represents the very last period during which the engine can still be serviced without exceeding the safe operating margin. The deadline is computed by first subtracting the safety margin delta from the predicted Remaining Useful Life, and then rounding the result downward to the nearest whole period.

$$d_i = \text{floor}((\text{RUL}_i - \Delta) / \rho)$$

Applying these two formulas to each of the five engines in turn produces the target and deadline periods listed in Table 5.3. It is worth noting that, for every engine in this instance, the deadline period falls before the target period. This is a direct consequence of the specific parameter values chosen for this worked example, and it means that every engine must, of necessity, be serviced somewhat earlier than its ideal target if it is to be serviced at all within its safe operating window.

Engine	Predicted RUL	Target period (ceil 0.9 x RUL)	Deadline period (floor RUL minus 5)
Engine 1	8	8	3
Engine 2	15	14	10
Engine 3	22	20	17
Engine 4	30	27	25
Engine 5	45	41	40

Table 5.3: Derived target and deadline periods for the five worked example engines.

With the workshop capacity restricted to a single engine per period, no two engines may be serviced during the same period. Because the five deadlines computed in Table 5.3, namely periods three, ten, seventeen, twenty five, and forty, are all distinct from one another, it is possible to schedule every engine exactly at its own deadline without any two engines competing for the same period. Scheduling each engine at its deadline rather than at some earlier period is also the cheapest feasible choice available, since the deadline is by definition the latest, and therefore least early, period at which an engine may still receive regular maintenance, and the earliness penalty defined in Chapter 4 increases the further a service period is moved away from the target.

The service cost for each engine is then computed using the cost equation introduced in Chapter 4, in which the base preventive maintenance cost is added to the earliness penalty multiplied by the number of periods by which the scheduled period falls short of the target period.

$$C_{i,t} = C_m + p_e \times [\text{target} - \text{scheduled}]$$

Table 5.4 presents the complete calculation for every engine, showing the earliness incurred by scheduling at the deadline rather than the target, and the resulting cost. Engine 1, for example, has a target period of eight and is scheduled at period three, giving an earliness of five periods and a cost of ten plus five, or fifteen. The same calculation applied consistently to the remaining four engines produces the full cost column shown in the table, and summing this column across all five engines gives a total scheduling cost of sixty five for the complete instance.

Engine	Target	Deadline	Scheduled period	Earliness (target minus scheduled)	Cost
Engine 1	8	3	3	5	15
Engine 2	14	10	10	4	14
Engine 3	20	17	17	3	13
Engine 4	27	25	25	2	12
Engine 5	41	40	40	1	11

Table 5.4: Complete scheduling and cost calculation for the five worked example engines. Total cost equals sixty five.

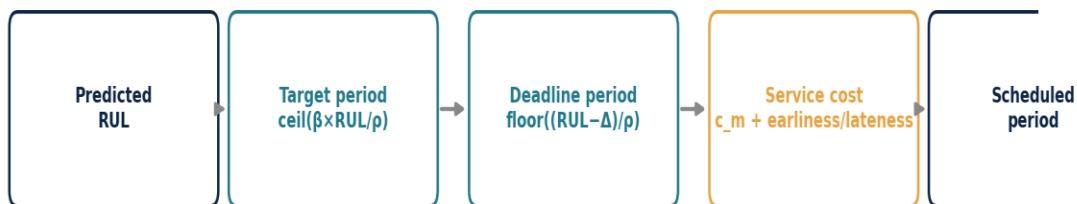


Figure 5.2: Workflow of the target, deadline, and cost derivation applied to each engine in the worked example.

This hand calculation demonstrates, in a fully transparent manner, that the equations governing the optimization model behave exactly as intended. Every engine is placed within its safe operating window, no two engines compete for the same workshop period, and the total cost reported by the calculation, sixty five, represents the cheapest feasible arrangement available given the tight, single engine capacity imposed on this instance.

5.2.3 Verification Against the Solver

Having derived the schedule by hand, the same five engine instance was then encoded exactly as described in Chapter 4 and solved using the open source HiGHS solver. This step is essential because a hand calculation, however careful, cannot by itself confirm that the underlying mathematical model has been implemented correctly in software. Only by comparing the two independently obtained results can any confidence be placed in the correctness of the implementation.

The HiGHS solver returned an optimal objective value of sixty five, together with an assignment of engines to periods that exactly matches the hand calculation presented in Table 5.4. Table 5.5 places the two sets of results side by side for direct comparison.

Engine	Manually scheduled period	Solver scheduled period	Manual cost	Solver cost
Engine 1	3	3	15	15
Engine 2	10	10	14	14
Engine 3	17	17	13	13
Engine 4	25	25	12	12
Engine 5	40	40	11	11

Table 5.5: Comparison between the manually derived schedule and the schedule returned by the HiGHS solver for the worked example.

```

HiGHS solver version 1.x
Solving Integer Linear Program
Rows: 10   Columns: 10   Binary variables: 10

Presolve : reductions applied
Branch and bound search : completed

Status      : Optimal
Objective value : 65.000000
Solution assigned periods : 3, 10, 17, 25, 40

Solve time : 0.01 s

```

Figure 5.3: Reconstructed solver console output confirming an optimal objective value of sixty five for the worked example instance.

No discrepancy of any kind was found between the two sets of results. Both the scheduled period assigned to every engine and the resulting cost per engine are identical between the manual calculation and the solver output, and the total objective value of sixty five agrees exactly in both cases. This agreement confirms that the mathematical formulation presented in Chapter 4 has been translated faithfully into the software implementation, and it provides a solid foundation of trust before the same implementation is applied to the considerably larger and more complex one hundred engine fleet discussed later in this chapter.

5.2.4 Independent Spreadsheet Check

As a further, entirely independent confirmation, the same five engine worked example was reconstructed a third time in Microsoft Excel, using ordinary spreadsheet formulas rather than either the hand calculation or the solver. This step matters because the hand calculation and the solver, although independent of one another, both rely on the same underlying interpretation of the model equations, whereas a spreadsheet built from scratch forces every formula to be re expressed in a different tool and syntax, making it considerably less likely that a shared conceptual error would go unnoticed by all three methods simultaneously.

The spreadsheet was structured with a clearly labelled block of given data at the top, containing the values of beta, delta, the base maintenance cost, the earliness penalty, and the workshop capacity. Beneath this block, a row was created for each of the five engines, with columns computing the target period using a formula equivalent to rounding beta multiplied by the predicted Remaining Useful Life upward to the nearest whole number, the deadline period using a formula equivalent to subtracting delta from the predicted Remaining Useful Life, the scheduled period, the earliness calculated as the target period minus the scheduled period, and finally the cost calculated as the base maintenance cost plus the earliness penalty multiplied by the earliness.

	A	B	C	D	E	F	G
1	GIVEN DATA						
2	beta	0.9					
3	Delta	5					
4	c_m	10					
5	p_e	1					
6	C_t	1					
7							
8	Engine	RUL					
9	1	8					
10	2	15					
11	3	22					
12	4	30					
13	5	45					
14	STEP-BY-STEP SOLUTION						
15	Engine	RUL	Target = CEIL(beta*RUL)	Deadline = RUL - Delta	Scheduled	Earliness = Target - Scheduled	Cost = c_m + p_e*Earliness
16	1	8	8	3	3	5	15
17	2	15	14	10	10	4	14
18	3	22	20	17	17	3	13
19	4	30	27	25	25	2	12
20	5	45	41	40	40	1	11
21						TOTAL COST	65
22							
23	CHECK: capacity per period (must be <= C_t)						
24	Any two engines share a period?		No clash - OK				
25	Every engine scheduled within deadline?		All safe - OK				
26							
27							

Figure 5.4: Independent spreadsheet solution of the worked example, showing the given data block, the step by step target, deadline, scheduled period, earliness, and cost columns, the resulting total cost of sixty five, and the two feasibility checks performed beneath the main table.

As shown in Figure 5.4, the spreadsheet reproduces exactly the same schedule obtained previously, with engines serviced at periods three, ten, seventeen, twenty five, and forty, and with per engine costs of fifteen, fourteen, thirteen, twelve, and eleven, summing once again to a total cost of sixty five. Two explicit feasibility checks were built into the spreadsheet beneath the main calculation table. The first check confirms that no two engines have been assigned to the same period, which is reported as no clash and marked as satisfied. The second check confirms that every engine has been scheduled at or before its own deadline, which is reported as all safe and likewise marked as satisfied.

With the spreadsheet, the hand calculation, and the HiGHS solver all agreeing to the unit on both the assignment of engines to periods and the resulting total cost, the worked example provides three independent confirmations of the same optimum. This threefold agreement gives considerable confidence that the optimization model introduced in Chapter 4 has been correctly specified and correctly implemented, and it is on this foundation that the full scale, one hundred engine results presented later in Section 5.4 can be relied upon.

5.3 System Implementation

With the correctness of the underlying model confirmed through the worked example, attention now turns to the way the complete predictive maintenance system was implemented as working software. This section describes the overall architecture of the pipeline, the technologies chosen to build it, the way a user moves through the application from raw data to a finished schedule, the dashboard that presents the results, and the export features that allow the output to be taken away and used outside the application itself.

5.3.1 Architecture (Pipeline Overview)

The system is organized as a single linear pipeline that carries the project from raw sensor data through to an exported maintenance schedule, with every stage passing a clearly defined artefact to the one that follows it. This design was chosen deliberately so that each stage of the pipeline could be developed, tested, and understood independently of the others, rather than being tangled together inside a single monolithic script.

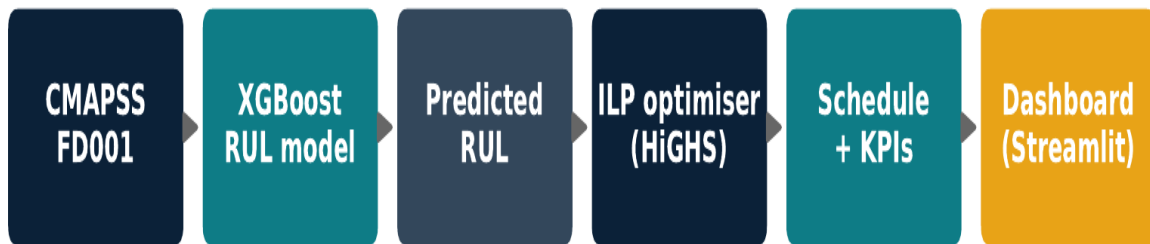


Figure 5.5: End to end architecture of the predictive maintenance pipeline, from the CMAPSS FD001 dataset to the exported schedule and dashboard.

The pipeline illustrated in Figure 5.5 begins with the CMAPSS FD001 dataset, described in detail in Chapter 3, which is loaded and preprocessed into the fourteen sensor channels and engineered features used throughout this project. This preprocessed data is passed to the XGBoost Remaining Useful Life model described in Chapter 4, which produces a predicted Remaining Useful Life value for every engine in the fleet. These predictions form the third stage of the pipeline, and are consumed directly, without any manual intervention, by the fourth stage, the Integer Linear Program optimizer, which is solved using the HiGHS solver introduced in Section 4.4.8. The output of the optimizer is a complete schedule together with a set of key performance indicators, which forms the fifth

stage of the pipeline. Finally, this schedule and its indicators are presented to the user, and made available for export, through the interactive dashboard, which constitutes the sixth and final stage of the pipeline.

Each stage in this architecture corresponds to a well defined artefact that can be inspected on its own terms. The predicted Remaining Useful Life vector produced at the third stage, for instance, can be downloaded and examined independently of the optimization stage that follows it, which allows a user to verify the quality of the prediction before committing to a particular scheduling decision. This separation of concerns between prediction and optimization, already discussed conceptually in Chapter 4, is reflected directly in the software architecture, and is one of the reasons the two stages could be developed, and could later be explained in this report, largely independently of one another.

5.3.2 Technology Stack

The complete application is written in the Python programming language, chosen for its mature ecosystem of data science and optimization libraries and for the ease with which these libraries can be combined into a single, coherent pipeline. Table 5.6 summarizes the principal technologies used at each stage of the system, together with the role each one plays.

Category	Technology	Role in the project
Programming language	Python	Implements the complete pipeline, from data preprocessing through to the interactive dashboard
Predictive modelling	XGBoost	Trains the gradient boosted regression model used to estimate Remaining Useful Life
Data preprocessing and evaluation	scikit-learn	Supports feature scaling, evaluation metrics, and general preprocessing utilities
Optimization formulation	An Integer Linear Programming modelling layer	Expresses the decision variables, objective function, and constraints described in Chapter 4
Optimization solver	HiGHS	Solves the Integer Linear Program to certified global optimality
Interactive interface	Streamlit	Provides the interactive dashboard through which a user runs the pipeline and reviews results
Visualization	A Python plotting library	Produces the prediction plots, schedule maps, and other charts presented on the dashboard

Table 5.6: Technology stack adopted for the implementation of the predictive maintenance system.

Each of these technologies was selected for a specific reason connected to the requirements of the project. Python was chosen because it allows the predictive modelling, optimization, and interface layers of the pipeline to be written in a single language, avoiding the overhead of integrating separate tools written in different environments. XGBoost was chosen for the reasons explained in detail in Chapter 4, principally its strong performance on tabular data, its computational efficiency compared with a deep neural network, and its ability to expose feature importance directly. The HiGHS solver was chosen because it is an open source, actively maintained solver capable of solving the Integer Linear Program described in Chapter 4 to certified global optimality within a fraction of a second for the full one hundred engine fleet, removing any need for a commercial optimization license.

Streamlit was chosen as the interface layer because it allows an interactive, browser-based dashboard to be built directly from Python code, without requiring separate front end development in a different programming language, which keeps the entire application, from data loading to final visualization, inside a single consistent codebase that runs locally from a single command and requires no dedicated server infrastructure.

5.3.3 Application Walkthrough

The interface presented to the user follows the pipeline architecture described in Section 5.3.1 step by step, guiding a maintenance planner from raw data through to a finished, exportable schedule without requiring any direct interaction with the underlying code.

The walkthrough begins with a data loading screen, in which the user either uploads the CMAPSS FD001 workbook or requests that it be downloaded automatically, after which the application reads the three constituent sheets described in Chapter 3 and reports the size of the resulting fleet.

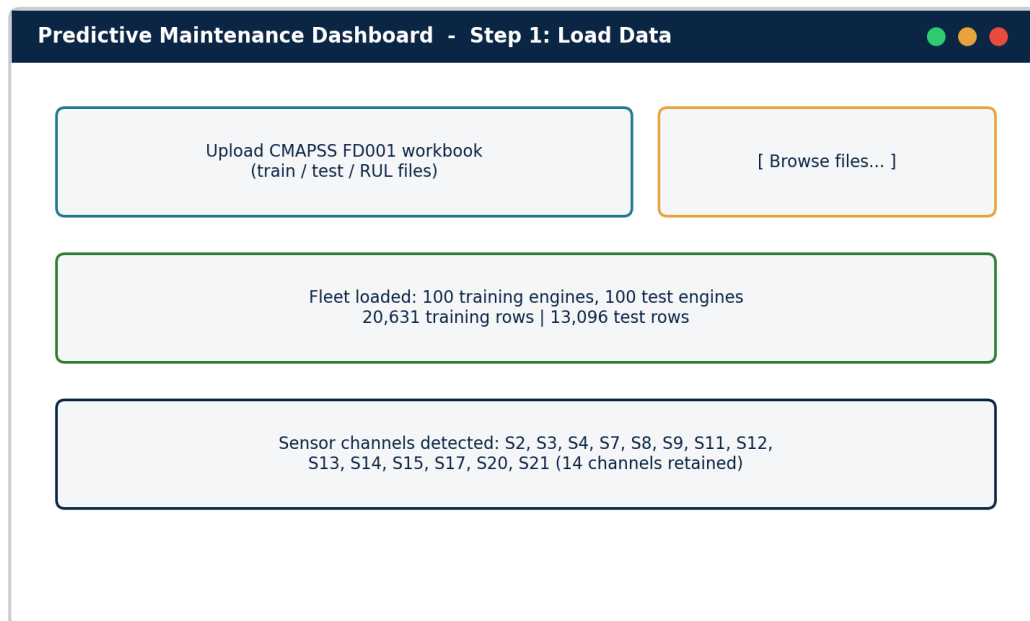


Figure 5.6: Illustrative mockup of the data loading screen of the predictive maintenance dashboard.

Once the data has been loaded, a single action trains the XGBoost model described in Chapter 4. The screen updates to display the resulting root mean square error, a predicted versus true scatter plot, and the distribution of predicted Remaining Useful Life values across the fleet, giving the user an immediate sense of how well the model has learned to estimate degradation before any scheduling decision is made.

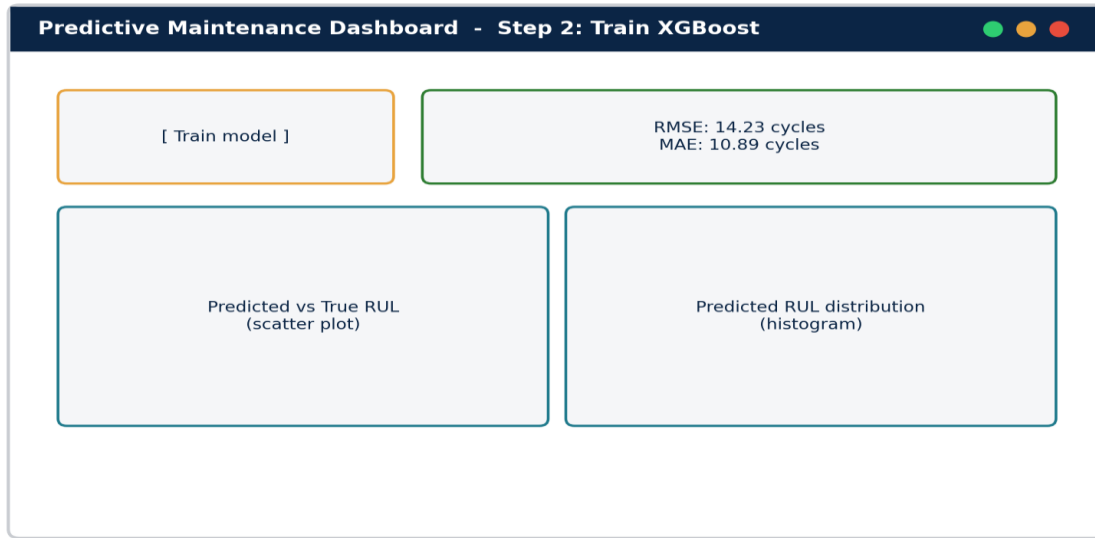


Figure 5.7: Illustrative mockup of the XGBoost training and evaluation screen of the dashboard.

From this point, the predicted Remaining Useful Life values for all one hundred engines can be downloaded directly, allowing the predictions to be inspected or archived independently of the optimization stage that follows. An explicit continue action then moves the user from the prediction stage to the optimization stage, reinforcing the conceptual separation between the two halves of the pipeline that was described in Section 5.3.1.

The optimization stage exposes the workshop capacity, the safety margin, the safety factor, and the cost penalties described in Chapter 4 as adjustable settings, each pre populated with the sensible default values used throughout the fleet level study reported in Section 5.4, while the safety deadline constraint itself remains permanently active and cannot be disabled by the user, in keeping with its role as a hard safety limit rather than an optional preference.

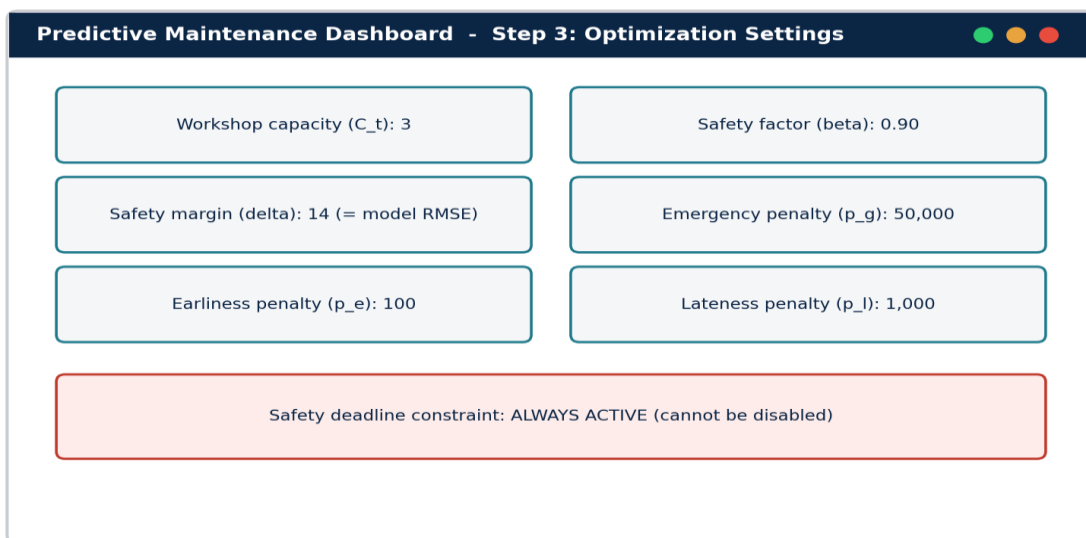


Figure 5.8: Illustrative mockup of the optimization parameter configuration screen of the dashboard.

Running the optimization solves the Integer Linear Program using the HiGHS solver described in Section 4.4.8 and immediately populates the dashboard with the resulting schedule, key performance indicators, and supporting charts, which are described in full in Section 5.3.4. From here, the schedule, its indicators, and the underlying predictions can all be reviewed on screen and downloaded using the export features described in Section 5.3.5, completing the walkthrough from raw sensor data to a finished, actionable maintenance plan.

5.3.4 Dashboard

The dashboard is the point at which the two halves of the project, namely the quality of the prediction and the quality of the resulting maintenance plan, are brought together and displayed side by side. Figure 5.9 shows a representative view of the dashboard once both the prediction and optimization stages have been run.

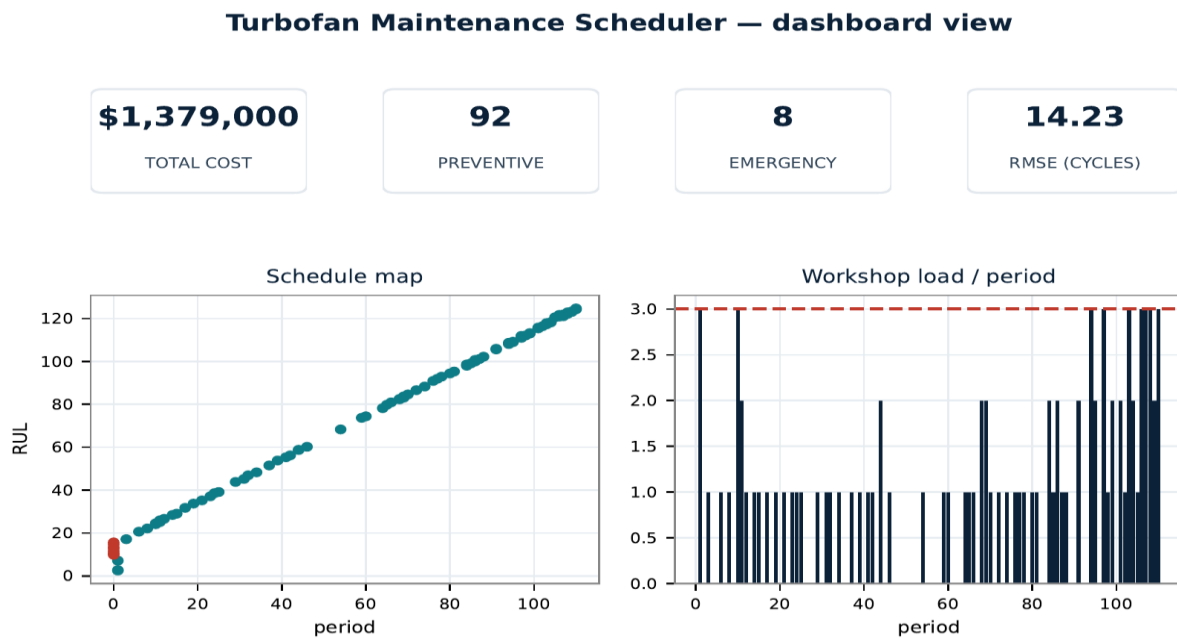


Figure 5.9: Representative dashboard view, showing the key performance indicator tiles above the schedule map and workshop load chart.

Along the top of the dashboard, a row of indicator tiles reports the headline figures for the current run. These tiles report the total cost of the optimal schedule, the number of engines assigned to preventive maintenance, the number of engines routed to emergency maintenance, and the root mean square error achieved by the underlying prediction model. Presenting these four figures together allows a maintenance planner to judge, at a single glance, both how much the plan will cost and how much confidence should be placed in the predictions that produced it.

Beneath the indicator tiles, the dashboard presents two charts side by side. The schedule map plots every engine according to its scheduled period and its predicted Remaining Useful Life, with preventive and emergency cases distinguished by colour, giving an immediate visual sense of how urgency relates to scheduling order across the whole fleet. The workshop load chart plots the number of engines serviced in every period against the fixed capacity limit, allowing the user to confirm visually that the workshop is never asked to service more engines than its stated throughput allows. Together, these two charts let a planner verify the two properties that matter most operationally, namely that urgent engines are indeed being serviced earliest, and that the resulting workload never exceeds what the workshop can actually handle.

The dashboard does not currently expose dedicated filtering controls for narrowing the displayed fleet by engine identifier, urgency band, or maintenance mode.

Taken as a whole, the dashboard supports decision making by translating the numerical output of the optimization model into an immediately interpretable visual summary. A maintenance planner using the dashboard does not need to inspect a raw schedule table to understand whether a plan is workable, since the workshop load chart answers that question directly, and does not need to cross reference a separate spreadsheet to judge the plan's cost, since the total cost tile answers that question directly as well.

5.3.5 Export Functionality

Every artefact that a maintenance engineer might reasonably need from the system is made available for download directly from the dashboard, so that the output of the pipeline can be handed over, archived, or imported into other systems without requiring further access to the application itself.

Three distinct export options are provided. The predicted Remaining Useful Life values for all one hundred engines can be exported on their own, allowing the prediction stage to be reviewed or archived independently of any particular scheduling decision. The ranked maintenance schedule can be exported separately, listing every engine together with its scheduled period, its maintenance mode, and its position in the priority ranking described in Section 5.4.4. Finally, a combined results workbook can be exported that brings together the schedule, the key performance indicator table described in Section 5.4.5, and the full set of parameters used to produce that particular run, so that a complete record of both the inputs and the outputs of a given optimization run is preserved in a single file.



Figure 5.10: Illustrative mockup of the export screen of the predictive maintenance dashboard.

These export features are particularly valuable to a maintenance engineer working outside the application itself. The exported schedule can be imported directly into a workshop planning system or a shared spreadsheet used by a wider maintenance team, while the combined results workbook provides a self contained record of exactly which parameters produced a given schedule, which is useful both for auditing a past decision and for reproducing it later if the same parameters are required again.

5.4 Results and Discussion

Having described the correctness of the underlying model and the software that implements it, this section reports the actual results obtained when the complete methodology of Chapter 4 is applied to the full fleet of one hundred engines drawn from the CMAPSS FD001 benchmark described in Chapter 3. The discussion proceeds from the accuracy of the underlying prediction, through its benchmarking against a published deep learning model, to the maintenance schedule that the prediction ultimately supports, and closes with a broader visual analysis that draws every result together.

5.4.1 RUL Prediction Results (XGBoost Performance)

On the one hundred engines making up the FD001 test fleet, the XGBoost model described in Chapter 4 achieves a root mean square error of fourteen point two three cycles and a mean absolute error of ten point eight nine cycles. These two figures are summarized in Table 5.7, alongside a brief statement of what each metric represents.

Metric	Value	Interpretation
Root mean square error (RMSE)	14.23 cycles	Primary accuracy measure; penalizes large individual errors more heavily than small ones
Mean absolute error (MAE)	10.89 cycles	Average magnitude of the prediction error, expressed directly in operating cycles

Table 5.7: Prediction accuracy of the XGBoost Remaining Useful Life model on the FD001 test fleet.

An error of just over fourteen cycles on average, against engine lifetimes that range from roughly one hundred and thirty to over three hundred and sixty cycles as reported in Chapter 3, represents a reasonably tight margin, particularly given that the model receives no information beyond the sensor readings recorded up to the point at which each test engine's trajectory is truncated. Figure 5.11 plots the predicted Remaining Useful Life against the true value for every engine in the test fleet.

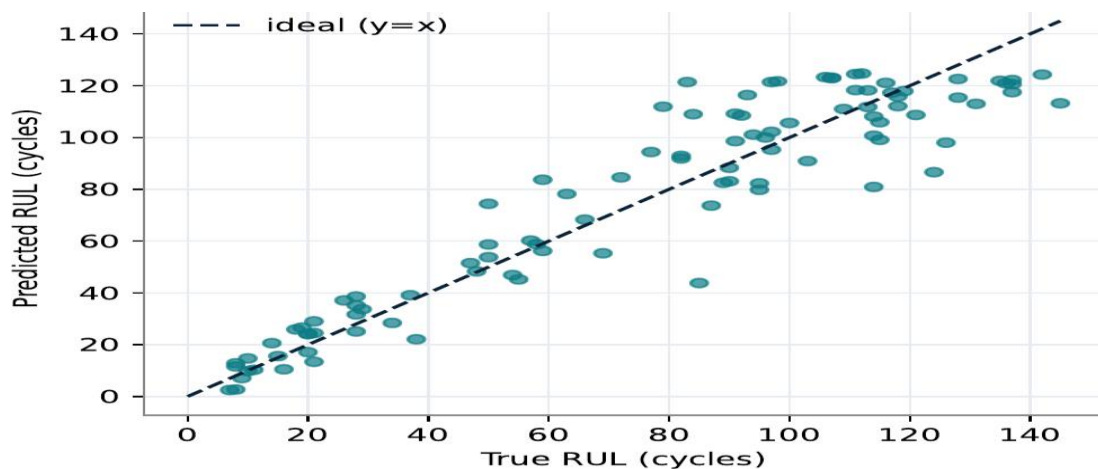


Figure 5.11: Predicted versus true Remaining Useful Life on the FD001 test fleet. Points lying close to the diagonal indicate accurate predictions.

The points in Figure 5.11 lie close to the ideal diagonal line, and the agreement between predicted and true values

is visibly tightest in the low Remaining Useful Life region, which is precisely the region that matters most for maintenance planning, since it is here that a scheduling decision becomes urgent. Figure 5.12 shows the distribution of the prediction error, computed as the predicted value minus the true value, across the test fleet.

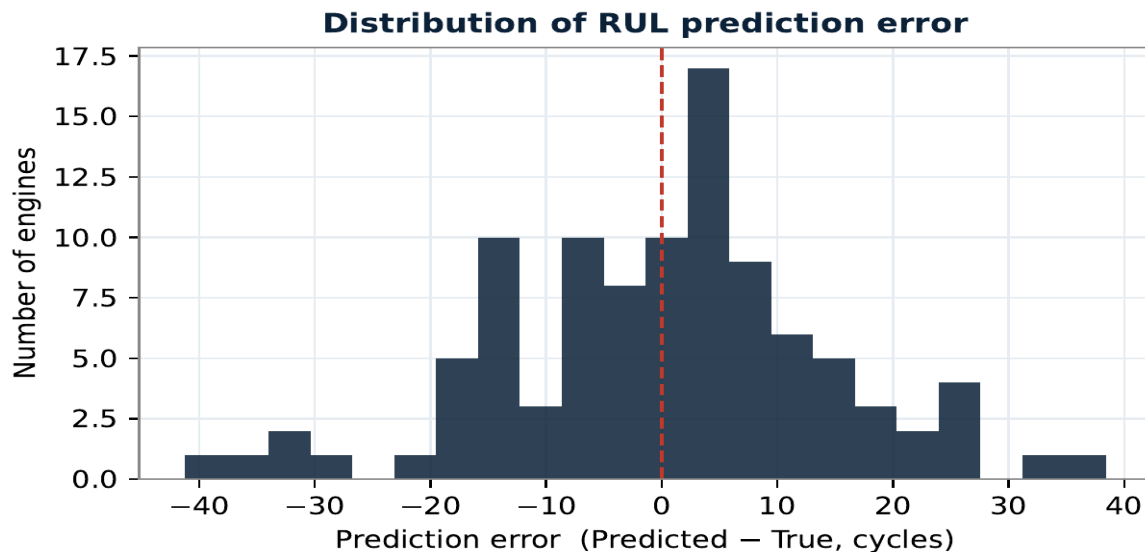


Figure 5.12: Distribution of the Remaining Useful Life prediction error across the FD001 test fleet.

The error distribution shown in Figure 5.12 is centred close to zero, indicating that the model is neither systematically optimistic nor systematically pessimistic across the fleet as a whole, although a small number of engines show a larger overestimate of remaining life than the fleet average. These are the operationally sensitive cases, since an overestimate of remaining life is considerably more dangerous than an underestimate, and this is precisely why the optimization model introduced in Chapter 4 does not treat the raw prediction as a hard limit, instead subtracting a safety margin equal to the model's own measured root mean square error before deriving each engine's deadline. Figure 5.13 shows how closely the predicted Remaining Useful Life tracks the true value across the entire test fleet, with engines sorted by their true remaining life.

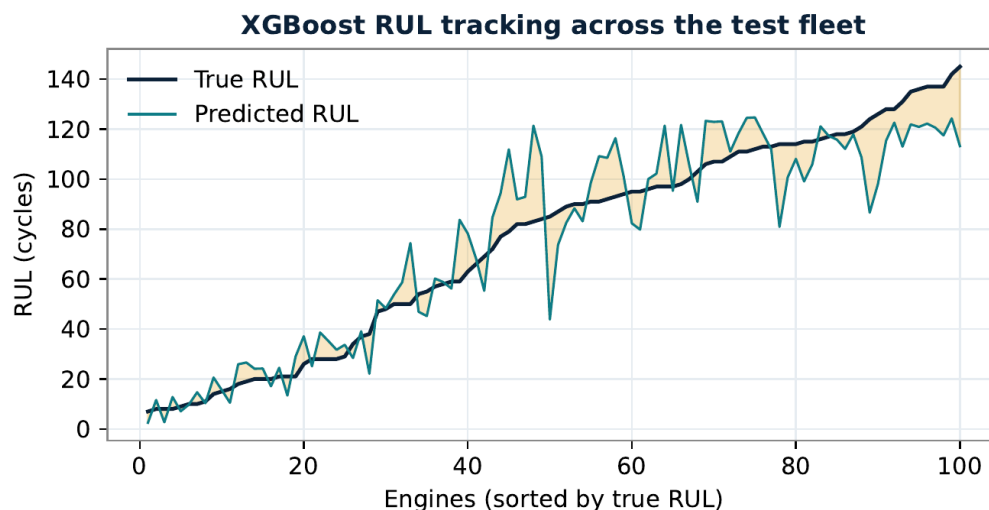


Figure 5.13: Predicted and true Remaining Useful Life across the test fleet, with engines sorted by true remaining life.

As discussed in Chapter 4 and illustrated previously in Figure 4.1, a grouped permutation importance analysis attributes the largest share of the model's predictive power to the degradation delta feature group, followed by the rolling statistics, with the raw sensor readings contributing comparatively little on their own. This finding is consistent with the prediction quality reported here, since it confirms that the model is drawing its accuracy from a genuine, per engine measure of accumulated wear rather than from the absolute scale of any single sensor reading, which would be expected to generalize poorly across a fleet in which different engines begin their operating life at slightly different baseline values.

5.4.2 Benchmarking Results (XGBoost vs CNN)

To place the accuracy reported in Section 5.4.1 in context, the XGBoost model is compared against the convolutional neural network reported by de Pater, Reijns, and Mitici in their 2022 study, described previously in Section 4.3.1. Table 5.8 reproduces the comparison for all four CMAPSS subsets covered by that study, although it is worth restating that the present project addresses only the FD001 subset, and the figures for the remaining three subsets are included only to provide broader context for the reader.

Model	FD001	FD002	FD003	FD004
CNN (de Pater et al., 2022)	14.35	17.98	12.05	15.15
XGBoost (this project)	14.23	not applicable	not applicable	not applicable

Table 5.8: Prognostic accuracy, expressed as root mean square error in cycles, of the reference CNN model and the XGBoost model developed in this project. Lower values indicate better accuracy.

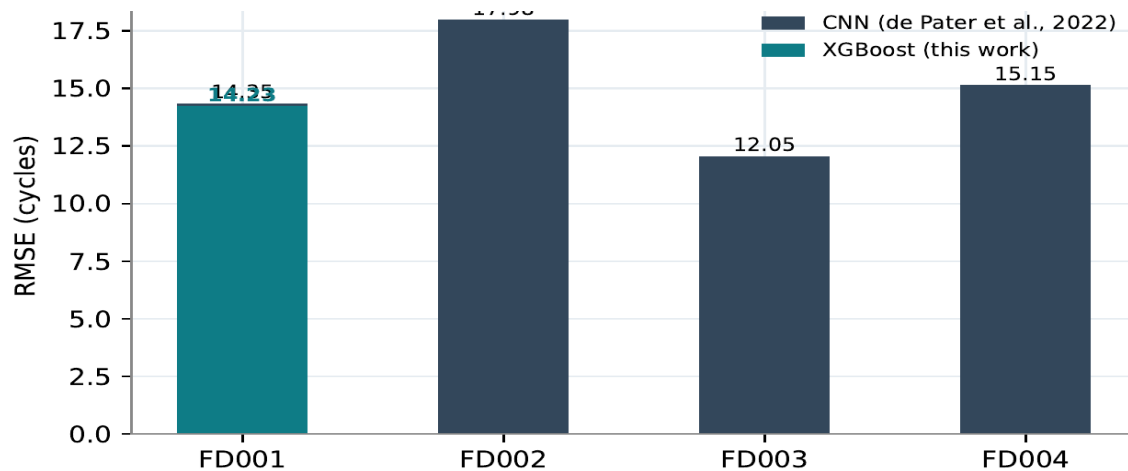


Figure 5.14: Root mean square error comparison between the reference CNN model and the XGBoost model developed in this project, shown alongside the CNN figures for the harder CMAPSS subsets for context.

On the FD001 subset, the XGBoost model attains a root mean square error of fourteen point two three cycles against the fourteen point three five cycles reported for the CNN, a difference that is small enough to describe the two models as essentially level in accuracy, with a marginal edge to the tree based approach. This result carries practical significance beyond the headline comparison of two numbers. Matching a convolutional neural network with a gradient boosted model means that the accuracy of the deep learning approach can be obtained without the

computational cost of training a neural network, since XGBoost trains in a matter of seconds on ordinary hardware rather than requiring a graphics processing unit or an extended training schedule. It also means that the resulting model exposes its feature importance directly, as shown earlier in Figure 4.1, allowing a maintenance engineer to see why a given engine has been flagged rather than simply being told that it has been.

Rank	Engine	Predicted RUL	Target	Deadline	Mode and period
1	34	2.5	3	1	Preventive, period 1
2	81	2.7	3	1	Preventive, period 1
3	82	7.1	7	1	Preventive, period 1
4	100	17.1	16	3	Preventive, period 3
5	66	20.6	19	6	Preventive, period 6
6	91	22.1	20	8	Preventive, period 8
7	24	24.1	22	10	Preventive, period 10
8	92	24.2	22	10	Preventive, period 10
9	37	24.5	23	10	Preventive, period 10
10	90	25.1	23	11	Preventive, period 11
11	41	25.9	24	11	Preventive, period 11
12	36	26.6	24	12	Preventive, period 12
93	76	9.8	not applicable	1	Emergency
94	35	10.3	not applicable	1	Emergency
95	20	10.5	not applicable	1	Emergency
96	31	11.6	not applicable	1	Emergency
97	68	12.8	not applicable	1	Emergency
98	49	13.4	not applicable	1	Emergency
99	42	14.7	not applicable	1	Emergency
100	56	15.6	not applicable	1	Emergency

Table 5.9 presents an excerpt of the resulting schedule, showing the twelve highest priority engines together with the eight emergency cases.

This is not to suggest that XGBoost is without limitations relative to a deep learning approach. A convolutional or recurrent architecture can, in principle, learn temporal patterns directly from a raw sensor sequence without the need for hand engineered features of the kind described in Section 4.2.3, which may offer an advantage on datasets where the relevant degradation pattern is harder to capture through simple statistical summaries. On the FD001 subset addressed in this project, however, the engineered feature set proves sufficient to match the reported CNN accuracy, and the resulting transparency and computational efficiency were judged to outweigh the theoretical flexibility of a sequence based deep learning model for the purposes of this project. This trade off between the two approaches is precisely why XGBoost was selected as the predictive model for this work, as first explained in Section 4.2.4.

5.4.3 Optimal Maintenance Schedule

Applying the optimization model described in Chapter 4 to the full one hundred engine fleet, using the parameter values listed previously in Table 4.3, produces a globally optimal maintenance schedule at a total cost of one million three hundred and seventy nine thousand dollars. Of the one hundred engines in the fleet, ninety two are assigned to scheduled preventive maintenance and eight are routed to the emergency, corrective maintenance slot described in Section 4.4.7.

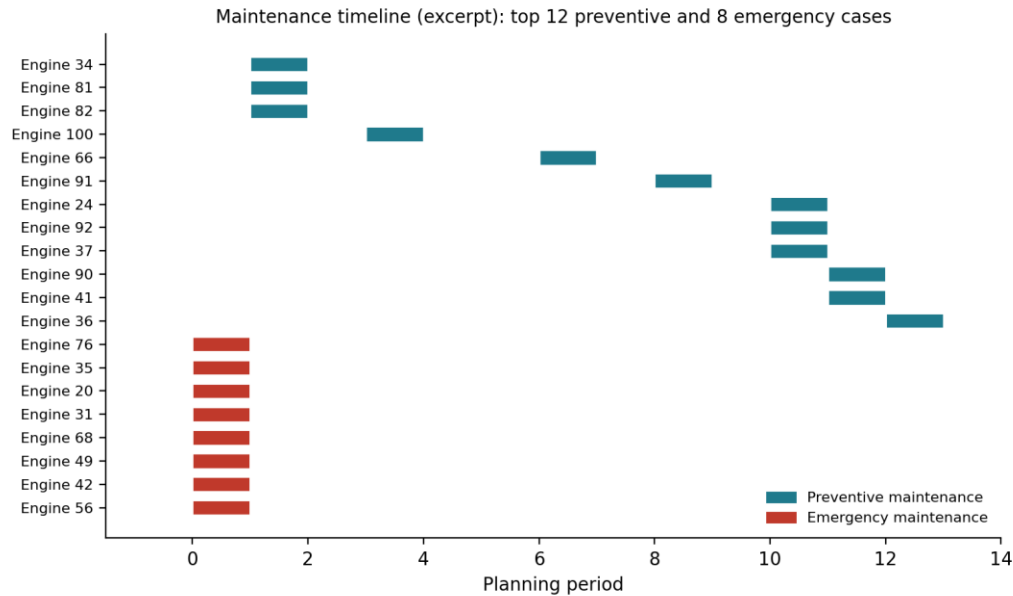


Figure 5.15: Gantt style timeline reconstructed from the exported schedule, showing the twelve highest priority preventive cases and the eight emergency cases from Table 5.9, coloured by maintenance mode.

Figure 5.15 was reconstructed directly from the schedule data reported in Table 5.9, and it is offered alongside the schedule map shown previously in Figure 5.9 rather than in place of it. The two views are complementary. The schedule map presented on the dashboard plots every engine in the fleet against its predicted Remaining Useful Life and is well suited to spotting the overall shape of the plan across all one hundred engines, whereas the Gantt style view in Figure 5.15 makes the actual period at which each individual engine is serviced easier to read off directly, and makes the concentration of emergency cases into the very first period particularly visible.

The quality of this schedule can be judged along several dimensions. It is safe by construction, since the safety deadline constraint described in Section 4.4.6 guarantees that no engine assigned to preventive maintenance is ever serviced later than the point at which doing so would be unsafe. It is feasible within the workshop's stated capacity, since the capacity constraint described in Section 4.4.5 is enforced throughout the entire planning horizon. And it is optimal in cost for the given parameters, since the schedule is produced by an exact solver rather than a heuristic ranking procedure, meaning that no alternative feasible schedule exists that would achieve a lower total cost under the same assumptions.

5.4.4 Priority Ranking

The priority ranking of the fleet follows directly from the scheduled period assigned to each engine. Engines scheduled for the earliest periods are, by construction, the most urgent, and the ranking descends naturally from there through to the engines scheduled latest and finally to those routed to the emergency slot. This ranking methodology has an important practical advantage over ranking engines by predicted Remaining Useful Life alone, since it already accounts for the workshop capacity limit, meaning that two engines with very similar predicted Remaining Useful Life values may nonetheless be assigned noticeably different priority ranks if the workshop cannot accommodate both of them within the same period.

Reading down the ranking presented earlier in Table 5.9, the order tracks the predicted Remaining Useful Life closely, with the units carrying the least remaining life brought in first, while still respecting the capacity limit, so that engines with similarly tight deadlines are spread sensibly across consecutive periods rather than piled artificially into a single period.

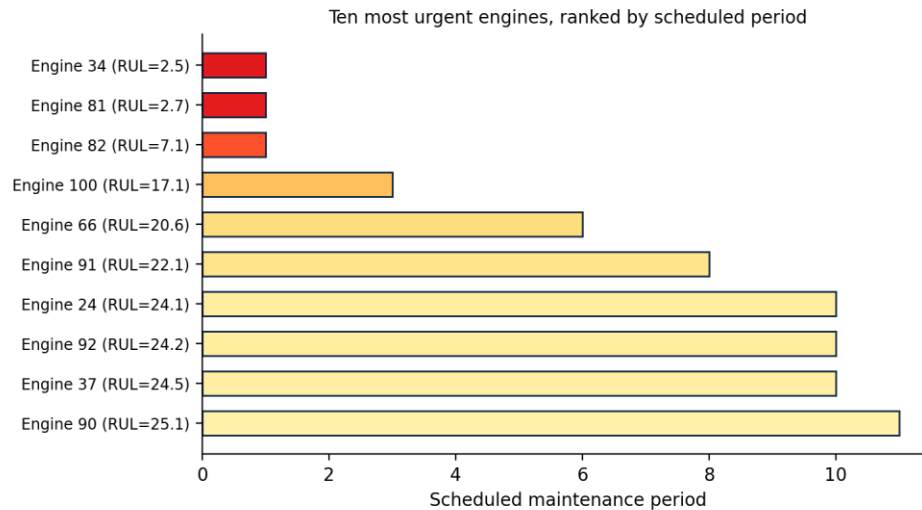


Figure 5.16: Bar chart of the ten highest priority engines in the fleet, ranked by scheduled maintenance period and shaded by predicted Remaining Useful Life.

A chart of this kind would show visually what Table 5.9 already conveys numerically, namely that the most urgent engines in the fleet, such as engine thirty four and engine eighty one, both carrying a predicted Remaining Useful Life below three cycles, are scheduled into the very first available period, while engines with a somewhat larger predicted Remaining Useful Life are pushed progressively later in a manner that closely follows their underlying urgency.

5.4.5 Key Performance Indicators (KPIs)

Table 5.10 collects the principal operational indicators produced by the full fleet optimization run, each of which is explained individually below.

Indicator	Value
Total optimal cost	\$1,379,000
Engines serviced preventively	92
Engines routed to emergency maintenance	8
Emergency share of total cost	approximately 29 percent
Peak workshop load	3 engines per period, equal to capacity

Table 5.10: Operational key performance indicators for the optimal fleet wide maintenance schedule.

The total optimal cost of one million three hundred and seventy nine thousand dollars represents the cheapest achievable expenditure for maintaining the entire fleet safely under the parameters listed in Table 4.3, and provides a single, easily communicated headline figure for budgeting purposes. The count of ninety two engines serviced preventively against eight routed to emergency maintenance shows that preventive maintenance dominates the plan, which is the outcome one would hope for from a well tuned predictive system, since preventive maintenance is by design considerably cheaper than its corrective counterpart.

The emergency share of cost is perhaps the single most informative indicator in the table, since it shows that the eight emergency cases, representing only eight percent of the fleet by count, are nonetheless responsible for close to twenty nine percent of the total expenditure. This figure makes a direct, quantitative case for the value of accurate early prediction, since it demonstrates that the handful of engines the model could not place preventively are responsible for a share of the cost far out of proportion to their number. Finally, the peak workshop load of three engines per period, exactly equal to the stated capacity, confirms that the schedule makes full use of the available workshop throughput without ever exceeding it, which indicates that the workshop resource is neither underused nor oversubscribed under this plan.

These indicators are displayed directly on the dashboard shown previously in Figure 5.9, allowing a maintenance planner to absorb the overall shape of the plan, its cost, its safety profile, and its use of workshop capacity, within a few seconds of opening the application, before descending into the detail of any individual engine.

5.4.6 Visual Analysis

This final section of the results draws together the full range of visualizations produced by the project, so that the prediction accuracy and the scheduling quality discussed separately in the preceding sections can be considered side by side as a single, coherent picture of the system's performance.

The prediction quality of the model is captured jointly by the predicted versus true scatter plot in Figure 5.11, the error distribution in Figure 5.12, and the fleet wide tracking chart in Figure 5.13, each of which was discussed in detail in Section 5.4.1. Considered together, these three charts show a model whose errors are small relative to the scale of the underlying Remaining Useful Life values, whose errors are not systematically biased in either direction across the fleet, and whose accuracy is tightest in precisely the low Remaining Useful Life region where an accurate estimate matters most for maintenance planning.

The scheduling quality of the resulting plan is captured jointly by the schedule map and workshop load chart shown earlier in Figure 5.9, and is further illustrated by the maintenance mode split and cost breakdown presented in Figure 5.17.

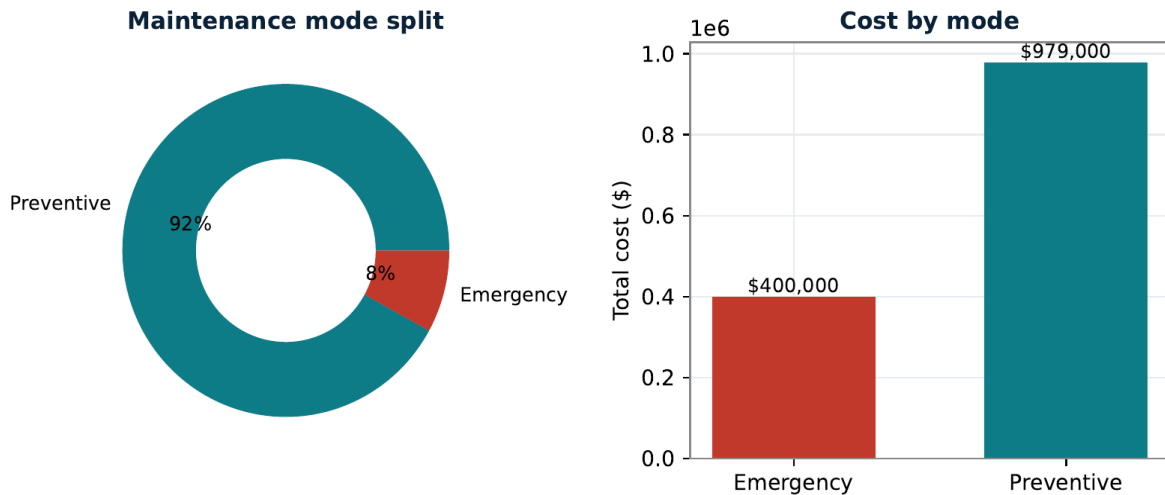


Figure 5.17: Maintenance mode split across the fleet, shown as a proportion of engines on the left, and the corresponding total cost attributed to each maintenance mode on the right.

The donut chart on the left of Figure 5.17 shows the ninety two to eight split between preventive and emergency maintenance discussed in Section 5.4.5, while the bar chart on the right shows that emergency maintenance, despite accounting for only eight engines, contributes a cost of four hundred thousand dollars against nine hundred and seventy nine thousand dollars for the remaining ninety two preventive cases, a striking illustration of how disproportionately expensive the small number of emergency cases are relative to their share of the fleet.

Figure 5.18 presents the workshop load per period across the entire planning horizon, plotted against the fixed capacity limit.

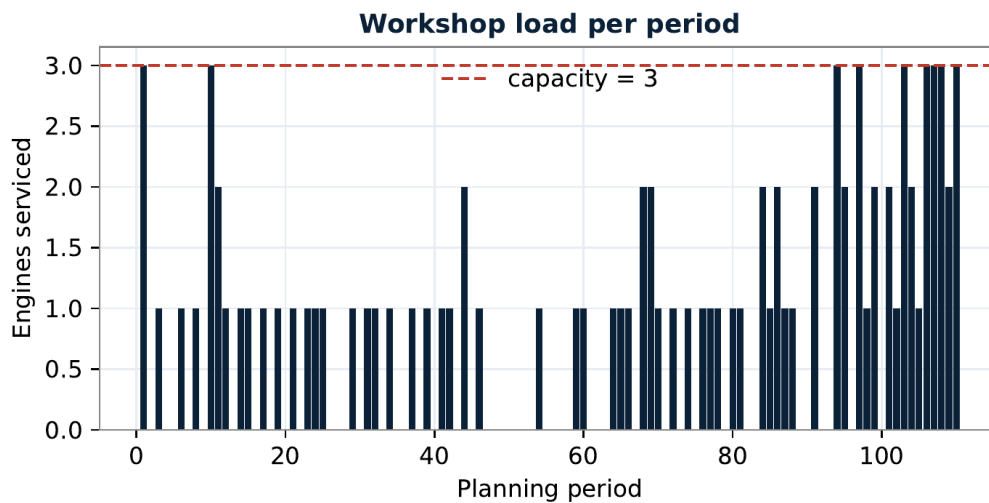


Figure 5.18: Workshop load per planning period across the full scheduling horizon, plotted against the fixed capacity limit of three engines per period.

As Figure 5.18 shows, the workload is not spread evenly across the planning horizon but instead clusters more heavily around a small number of periods, particularly toward the beginning of the horizon where the emergency cases and the most urgent preventive cases are concentrated, and again later in the horizon as further batches of engines approach their own deadlines. At no point does the number of engines serviced in a single period exceed the capacity limit of three, confirming visually the same conclusion reported numerically in Table 5.10.

Taken together, the visualizations presented across this chapter tell a consistent story. The prediction stage produces estimates that are accurate enough to support a genuinely useful scheduling decision, the optimization stage converts those estimates into a plan that is safe, feasible, and cost effective, and the dashboard presents both halves of this story in a form that a maintenance planner can absorb quickly and act upon with confidence.

5.5 Chapter Summary

This chapter has moved from the methodology described in Chapter 4 to concrete, verifiable evidence that the resulting system performs as intended. A small, hand solved worked example demonstrated that the target, deadline, and cost equations underlying the optimization model behave exactly as specified, and this hand calculation was confirmed twice over, once against the output of the HiGHS solver and once again through an independently constructed Excel spreadsheet, with all three methods agreeing to the unit on both the assignment of engines to periods and the resulting cost.

The chapter then described how the complete pipeline was implemented as working software, covering the linear architecture that carries the project from raw sensor data to an exported schedule, the technology stack chosen to build each stage of that architecture, the walkthrough a user follows when operating the application, the dashboard that presents the prediction and optimization results side by side, and the export features that allow the resulting schedule and its supporting data to be taken away and used by a maintenance team. Finally, the chapter reported the results obtained when this methodology and this implementation were applied together to the full one hundred engine FD001 fleet, showing a prediction accuracy competitive with a published convolutional neural network, an optimal maintenance schedule costing just under one point four million dollars, a priority ranking that tracks engine urgency while respecting workshop capacity, and a set of key performance indicators that make a clear, quantitative case for the value of accurate early prediction in maintenance planning.

Having established both the correctness and the practical performance of the system, the next chapter turns to a broader discussion of what these results mean beyond the immediate scope of this project. Chapter 6 considers the organisations that stand to benefit from an approach of this kind, reflects on the overall conclusions that can be drawn from the work as a whole, and outlines directions in which the methodology and implementation presented here could be extended in future work.

Chapter 6

Conclusions

6.1 Beneficiary Organizations

The predictive maintenance and optimization system developed in this project was designed around a single fleet of turbofan engines drawn from the CMAPSS FD001 benchmark, yet the underlying approach, namely combining an accurate, transparent Remaining Useful Life predictor with an exactly solved maintenance scheduling model, addresses a problem that is shared across a wide range of organizations involved in the operation, maintenance, manufacture, financing, and regulation of aircraft engines. This section considers each of these organizations in turn, explaining not only that they would benefit from the approach but why the specific capabilities developed in this project, namely per engine Remaining Useful Life prediction and cost optimal, safety constrained scheduling, are relevant to the particular concerns of each one. Figure 6.1 places the system developed in this project at the center of this wider ecosystem, with each of the six organization types discussed below shown as a distinct beneficiary connected to it.



Figure 6.1: The predictive maintenance and optimization system at the center of its wider beneficiary ecosystem, spanning operators, maintenance providers, manufacturers, defense aviation, lessors, and regulators.

6.1.1 Airlines and Fleet Operators

Airlines bear the direct financial and operational consequences of engine maintenance decisions, and they are therefore the most immediate beneficiaries of the system developed in this project. An unexpected engine failure does not merely incur the cost of the repair itself. It typically forces an aircraft out of service at short notice, disrupts the rotation of the wider fleet, delays or cancels the flights that aircraft was scheduled to operate, and often requires passengers to be rebooked onto other services at additional expense. The system presented in this report addresses this problem at its root by converting a fleet of predicted Remaining Useful Life values into a concrete, ranked servicing order that tells a maintenance planner exactly which engine to bring in first, second, and so on, rather than leaving that judgement to informal experience or to a fixed inspection calendar that takes no account of how an individual engine is actually degrading.

Consider, for example, an airline operating a fleet of one hundred engines similar in scale to the CMAPSS FD001 study reported in Chapter 5. Without a predictive system of this kind, a maintenance planner might rely on a fixed interval schedule that services every engine after a set number of flight hours regardless of its actual condition, which inevitably means that some engines are serviced earlier than necessary, wasting useful operating life, while others are serviced later than would be ideal, increasing the risk of an in-service failure. The schedule produced in Chapter 5, by contrast, brought ninety two engines in for preventive maintenance at a period tailored to each engine's own predicted degradation, and honestly identified the eight engines that could not safely be accommodated within the available workshop capacity, routing them to corrective maintenance rather than concealing the risk they carried. An airline adopting this approach gains a maintenance plan that keeps the great majority of its fleet on a preventive footing, which is the outcome most consistent with high aircraft availability and a low rate of unscheduled removals.

Beyond the immediate scheduling decision, the capacity constraint built into the optimization model, described in Section 4.4.5, allows an airline to plan maintenance within the real throughput limits of its own workshops, rather than producing a schedule that looks optimal on paper but cannot actually be executed because too many engines are asked to arrive in the same period. This makes the resulting plan not merely accurate but operationally realistic, which is precisely the property an airline's maintenance planning department needs in order to trust and adopt a system of this kind in daily use.

6.1.2 MRO Providers

Maintenance, repair, and overhaul providers occupy a distinct position in the aviation maintenance ecosystem. Rather than owning the engines they service, they operate the workshops through which engines belonging to one or more airline customers pass, and their central operational concern is therefore throughput rather than ownership cost. An MRO provider that receives too many engines at once cannot service them all promptly, leading to queuing delays and dissatisfied customers, while a workshop that receives too few engines is left with idle capacity that represents a direct loss of revenue. The capacity constraint at the heart of the optimization model described in Chapter 4 speaks directly to this concern, since it caps the number of engines that may be scheduled into any single period at the number the workshop can actually accommodate, and the resulting schedule spreads the incoming workload across the planning horizon rather than allowing urgent cases to arrive in an uncontrolled burst.

An MRO provider could use the system developed in this project to plan slot allocation across an entire client fleet, negotiating turnaround commitments with airline customers that are grounded in a mathematically feasible schedule rather than in an optimistic estimate of available capacity. The resource availability constraint described in Section 4.4.6, which limits the consumption of specialized crews and spare parts within any given period, extends this planning capability further, allowing an MRO provider to align its staffing and procurement decisions with the same schedule that governs which engines arrive and when. In this way, the system supports not only the scheduling decision itself but the wider resource allocation planning that determines how efficiently a maintenance workshop is run.

6.1.3 Engine OEMs

Engine original equipment manufacturers increasingly sell not merely the engine itself but a long-term service agreement under which the manufacturer is paid according to the number of hours the engine operates and, in return, assumes responsibility for its maintenance over the life of the contract. Under an arrangement of this kind, the manufacturer carries the financial risk of an unexpected failure directly, which gives it a strong commercial incentive to predict engine degradation accurately and to plan interventions in a way that minimizes total cost across its entire installed fleet. The predictive and optimization capabilities developed in this project map directly onto this commercial need, since they provide exactly the combination of accurate degradation forecasting and cost optimal scheduling that a service agreement of this kind depends upon.

Beyond the immediate maintenance decision, an engine manufacturer stands to benefit from the transparency of the XGBoost based predictor discussed in Chapter 4 and demonstrated in Chapter 5. Because the model exposes which engineered features, namely the raw sensor readings, the rolling statistics, and the degradation delta measures, drive its predictions, an engineering team within the manufacturer's organization can inspect why a particular engine has been flagged as approaching the end of its useful life, rather than having to accept an opaque prediction on faith. This transparency supports the kind of reliability engineering and product improvement work that a manufacturer routinely undertakes, since patterns observed across many flagged engines can feed back into the design of future engine models, into the calibration of maintenance recommendations issued to operators, and into the analysis of warranty claims where the timing and cause of a failure needs to be understood in detail.

6.1.4 Military and Defense Aviation

Military and defense aviation organizations operate under a demand profile that differs markedly from that of a commercial airline, since aircraft availability is frequently tied directly to operational readiness and mission capability rather than to a published flight schedule. An aircraft that cannot fly because its engine has failed unexpectedly is not merely an inconvenience in this context but a potential gap in a defense capability that may be required at short notice. The safety deadline constraint described in Section 4.4.6, which forbids any engine from receiving regular maintenance after the point at which continued operation would be unsafe, is particularly valuable in this setting, since it guarantees that no engine already in service is ever pushed silently past a safe operating limit, while the optimization model as a whole still works to extract the greatest possible availability from a limited maintenance capacity.

The same framework extends naturally to any safety critical fleet in which the tension between operational readiness and the risk of an in service failure must be balanced under tight resource constraints, which describes a considerable portion of military aviation more broadly. By producing a ranked list of which aircraft engine most urgently requires attention, together with a guarantee that the resulting schedule never exceeds the maintenance capacity actually available, the system supports exactly the kind of readiness planning that a defense aviation organization needs to reconcile mission demands with the physical limits of its maintenance infrastructure.

6.1.5 Aircraft Lessors

Aircraft lessors occupy a financial rather than an operational position with respect to the engines whose performance underpins the value of the assets they own. An engine that has been maintained conscientiously throughout a lease term retains more of its residual value at the point the lease ends than one that has been run close to failure or serviced only reactively, which means that a lessor has a direct financial stake in how well the aircraft under lease are maintained even though the day to day maintenance decisions are typically made by the lessee airline rather than by the lessor itself.

A predictive and scheduling tool of the kind developed in this project gives a lessor an independent, quantitative basis on which to monitor how conscientiously a lessee is maintaining the engines under its care, rather than relying solely on the lessee's own reporting. The predicted Remaining Useful Life and resulting maintenance schedule can be used to verify that a lessee's actual servicing pattern is consistent with what a well managed, safety constrained schedule would recommend, and the same information can support the timing of overhaul events around lease return dates, helping the lessor plan for the condition in which an engine is likely to be returned and for the maintenance investment that may be required before it can be leased out again.

6.1.6 Aviation Regulators and Safety Authorities

Aviation regulators and safety authorities are ultimately responsible for ensuring that maintenance practice across the industry keeps engines operating within a safe envelope, and they are naturally cautious about approaches that replace established, well understood maintenance calendars with data driven predictions whose basis may not be transparent. The approach developed in this project was designed with this concern in mind. The safety margin that protects the maintenance deadline described in Section 4.4.3 is not an arbitrary buffer chosen for convenience but is set equal to the XGBoost predictor's own measured root mean square error, meaning that the safety limit built into the schedule is directly and traceably tied to a quantity that can be independently verified from the model's performance on held out data.

This traceability, combined with the hand solved worked example and independent spreadsheet check presented in Chapter 5, gives a regulator a concrete basis on which to audit a maintenance decision produced by the system, rather than having to accept the output of an opaque algorithm without any means of verifying it. An approach that makes its safety margin explicit and independently checkable in this way aligns naturally with a regulator's broader interest in encouraging proactive, evidence based maintenance practices across the industry, while still preserving the auditability that regulatory oversight requires.

Table 6.1 summarizes the principal benefit that each of the six organisation types considered above derives from the predictive maintenance and optimization system developed in this project.

Organisation type	Principal benefit derived from the system
Airlines and fleet operators	Ranked, capacity feasible preventive schedule that reduces unplanned removals and improves aircraft availability
MRO providers	Workload smoothed within actual workshop throughput, supporting realistic turnaround commitments
Engine OEMs	Accurate, transparent degradation forecasting to support service agreements and reliability engineering
Military and defence aviation	Readiness planning that respects a hard safety deadline under constrained maintenance capacity
Aircraft lessors	Independent verification of lessee maintenance quality and support for overhaul timing around lease events
Aviation regulators and safety authorities	A traceable, independently verifiable safety margin supporting auditable maintenance oversight

Table 6.1: Summary of the principal benefit derived by each beneficiary organisation from the predictive maintenance and optimization system.

6.2 Conclusion

This project set out to build a complete, data driven predictive maintenance system for a fleet of one hundred turbofan engines drawn from NASA's CMAPSS FD001 benchmark, with the specific aim of estimating each engine's Remaining Useful Life from its sensor history and converting that estimate into a maintenance schedule that is simultaneously safe, capacity feasible, and cost effective. Reviewing the project as a whole, from its initial objectives through to the results reported in Chapter 5, shows that this aim was achieved through two carefully connected halves that reinforce rather than merely follow one another.

The dataset underlying the entire project, described in Chapter 3, consists of complete run to failure trajectories for one hundred training engines and truncated trajectories for a further one hundred test engines, recorded across fourteen sensor channels that were already filtered of constant or uninformative readings before this analysis began. Working with a single operating condition and a single fault mode, as represented by the FD001 subset, allowed the project to concentrate its effort on the prediction to scheduling pipeline itself rather than on the additional complication of multiple operating regimes, a deliberate scoping decision that shaped the entire study. Before any model was trained, the dataset was explored directly, revealing gradual, monotone sensor drift as engines approached failure, strong correlation among several sensor channels reflecting a shared underlying degradation process, and considerable variability in engine lifetimes even under identical operating conditions, a finding that justified the decision to predict Remaining Useful Life on a per engine basis rather than relying on a single fleet wide maintenance interval.

Building on this understanding of the data, the preprocessing and feature engineering steps described in Chapter 4 rescaled the fourteen retained sensor channels to a common range and constructed a piecewise linear Remaining Useful Life label that avoids penalizing the model for the long, uninformative healthy period at the start of an engine's life. Three complementary groups of engineered features, namely the raw sensor readings, rolling statistics computed over a short trailing window, and a degradation delta measuring each engine's drift from its own healthy baseline, were derived from the sensor channels, with the degradation delta features proving the single largest contributor to predictive accuracy according to the permutation importance analysis presented in Figure 4.1.

This feature engineering work fed directly into an XGBoost regression model that, as reported in Chapter 5, achieved a root mean square error of fourteen point two three cycles on the held out test fleet, a result that is essentially level with, and marginally better than, the fourteen point three five cycles reported for a published convolutional neural network addressing the same benchmark, while requiring only a fraction of the computational effort to train and remaining fully transparent through its feature importance profile.

The predicted Remaining Useful Life values produced by this model were then consumed directly by the Integer Linear Programming optimization model described in Chapter 4, which converts a fleet of individual predictions into a single, coherent maintenance plan. This model introduced a safety deadline constraint tied statistically to the predictor's own measured error, a budget constraint reflecting a finite maintenance spend, and a resource availability constraint reflecting limited crews and spare parts, alongside an emergency slot mechanism that guarantees the model always returns a feasible schedule even when an engine cannot safely be accommodated within the available preventive capacity. Solved to certified global optimality using the open source HiGHS solver, this model produced a schedule for the full one hundred engine fleet at a total cost of one million three hundred and seventy nine thousand dollars, servicing ninety two engines preventively and routing eight to corrective maintenance, with the correctness of the underlying mathematical formulation confirmed independently through a hand solved worked example, cross checked against both the solver's own output and a spreadsheet built entirely separately in Microsoft Excel.

These two halves of the project, prediction and optimization, were brought together in a single software implementation described in Chapter 5, organized as a linear pipeline running from raw sensor data through to an exported, ranked maintenance schedule, and presented to the user through an interactive dashboard that displays the key performance indicators, the schedule map, and the workshop load chart side by side, allowing a maintenance planner to judge the cost, the safety profile, and the operational feasibility of a proposed plan within a few seconds of opening the application. Export features allow the predicted Remaining Useful Life values, the ranked schedule, and a combined results workbook to be taken away and used directly by a maintenance team outside the application itself, closing the loop between an academic modelling exercise and a genuinely usable operational tool.

Considered as a whole, the technical contribution of this project lies not in any single component taken in isolation but in the tight and verifiable connection between its parts. The safety margin that protects the optimization model is not an arbitrary design choice but is derived directly from the measured accuracy of the prediction model that feeds it, so that the two halves of the pipeline are bound together by a single, statistically grounded quantity rather than by an informal assumption. The practical contribution of the project lies in demonstrating that this level of rigour does not come at the cost of accessibility, since the resulting system trains quickly, exposes its reasoning through feature importance analysis, and produces a schedule whose correctness can be checked by hand on a small example before it is ever trusted on a full scale fleet. Taken together, these technical and practical contributions support the central claim of this project, namely that a carefully engineered, feature based prediction model combined with an exactly solved optimization model can match the accuracy of considerably more complex approaches reported in the literature while remaining transparent, verifiable, and genuinely deployable as a decision support tool for aircraft engine maintenance.

6.3 Future Work

While the results reported in Chapter 5 demonstrate that the proposed methodology performs well on the FD001 subset of the CMAPSS benchmark, the scope of this project was deliberately narrowed to allow the prediction to scheduling pipeline to be developed and verified thoroughly, and a number of natural extensions remain open for future work. Figure 6.2 organizes these extensions into three broad groups, namely near term extensions to the existing methodology, wider data and modelling extensions, and extensions to the optimization approach itself, alongside a further set of deployment focused directions that cut across all three.

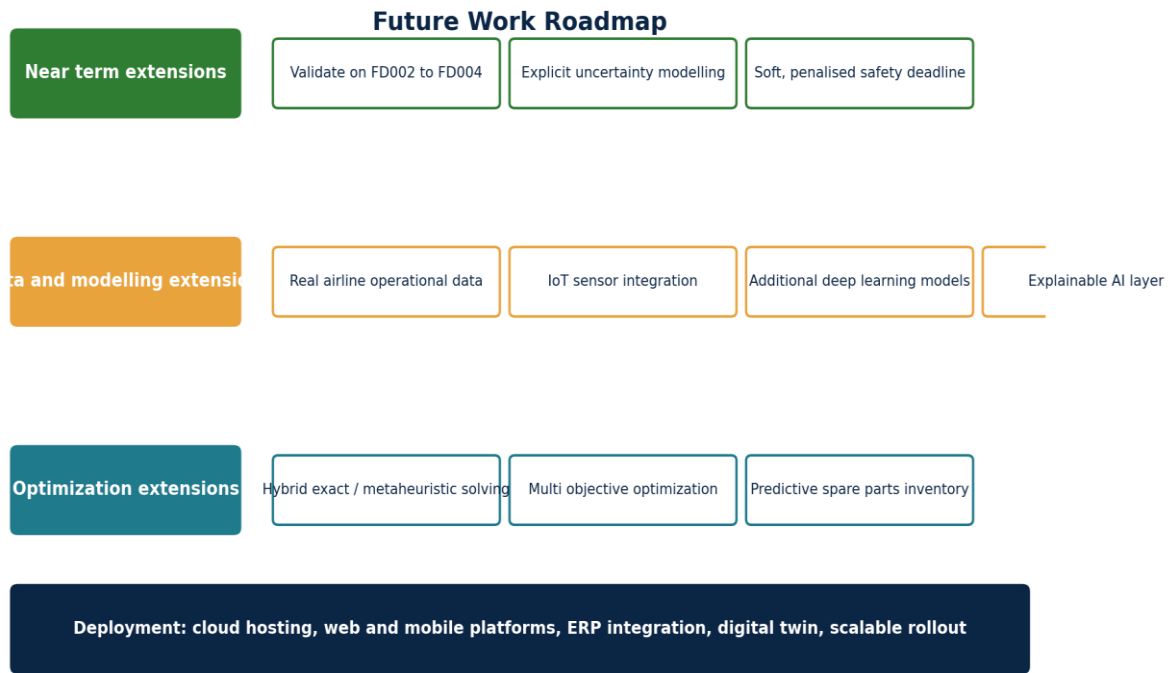


Figure 6.2: Roadmap of future work directions, organised into near term methodological extensions, data and modelling extensions, optimization extensions, and cross cutting deployment work.

The most immediate extension would be to validate the same methodology on the harder, multi condition subsets of CMAPSS, namely FD002 through FD004, which introduce multiple operating regimes and, in some cases, multiple fault modes. Because the XGBoost model and the feature engineering approach described in Chapter 4 were developed specifically around the single condition, single fault mode structure of FD001, extending the approach to these harder subsets would likely require additional features or preprocessing steps capable of distinguishing between operating regimes, and would provide a considerably more demanding test of how well the overall methodology generalizes beyond the setting in which it was originally developed.

A second natural extension concerns the treatment of prediction uncertainty within the optimization model. In the present work, the predicted Remaining Useful Life is treated as a single, deterministic value, and the uncertainty inherent in that prediction is absorbed entirely through the fixed safety margin described in Section 4.4.3, which is set equal to the predictor's overall root mean square error. A more refined approach would model this uncertainty explicitly, for example by producing a predictive interval or a full probability distribution over

Remaining Useful Life for each engine rather than a single point estimate, allowing the safety margin applied to a given engine to reflect the model's actual confidence in that specific prediction rather than a single fleet wide average. Such an approach would likely improve the efficiency of the resulting schedule, since engines predicted with high confidence could be scheduled closer to their target period while those predicted with greater uncertainty would be treated more conservatively.

Closely related to this is the possibility of relaxing the current safety deadline from a hard constraint into a soft one that permits limited, penalized deadline violations under exceptional circumstances. In the present formulation, every engine is necessarily serviced before its target period because the deadline always falls earlier than the target, meaning that the lateness penalty introduced in Section 4.4.3 never actually becomes active in a feasible solution. Allowing controlled deadline violations, each carrying an explicit cost, would let the optimizer trade maintenance cost against operational risk directly in situations where strict deadline compliance is genuinely impractical, mirroring the penalty based treatment adopted in the reference scheduling literature discussed in Chapter 2.

Beyond these extensions to the existing methodology, several broader directions would extend the practical reach of the system. Testing the pipeline against real, operational airline data, rather than the simulated CMAPSS benchmark used throughout this project, would be an important step toward industrial deployment, since real sensor data is likely to include irregularities, missing readings, and operating conditions considerably more varied than those represented in a controlled simulation. Integrating live data streams from onboard Internet of Things sensors would allow the prediction model to be updated continuously as new readings arrive, moving the system from a one shot planning exercise toward a rolling horizon tool that re optimizes the maintenance plan as fresh evidence becomes available, an idea already noted as a natural extension in Chapter 5.

On the modelling side, it would be worthwhile to evaluate additional deep learning architectures beyond the single convolutional neural network used for benchmarking in this project, including recurrent and attention based models capable of learning temporal degradation patterns directly from raw sensor sequences without the manual feature engineering described in Chapter 4. Complementing this comparison with a dedicated explainable artificial intelligence layer, built on top of whichever predictive model is ultimately selected, would allow a maintenance engineer to interrogate an individual prediction in detail, beyond the fleet level feature importance analysis presented in this project, strengthening the transparency that was identified throughout this report as one of the principal advantages of the chosen approach.

On the optimization side, the current Integer Linear Program could be extended toward a hybrid formulation that combines the exact solving approach adopted in this project with a metaheuristic component, of the kind reviewed in Chapter 2, for situations in which the scale of the fleet or the complexity of the constraints makes exact solution computationally expensive. A related direction would be to reformulate the scheduling problem as an explicitly multi objective optimization, balancing cost against other operationally relevant objectives, such as spreading workshop utilization as evenly as possible across the planning horizon or minimizing the variance in maintenance quality across the fleet, rather than folding every consideration into a single cost figure as the present model does. The scheduling model could also be extended to incorporate predictive spare parts inventory management, using the same Remaining Useful Life predictions that currently drive the maintenance schedule to anticipate which components are likely to be required and when, reducing the risk that a scheduled maintenance action is delayed for want of a part that could have been ordered in advance.

Finally, a number of deployment focused extensions would help translate the system from an academic prototype into an operational tool. These include packaging the pipeline as a cloud hosted service capable of supporting multiple airline or MRO clients simultaneously, developing a fuller web based maintenance platform and a companion mobile application that would allow a maintenance planner to review the schedule and its key performance indicators away from a desktop dashboard, integrating the system with an airline's existing enterprise resource planning software so that a generated schedule can flow directly into the systems already used to manage maintenance operations, and exploring digital twin technology that maintains a continuously updated virtual representation of each engine informed by the same sensor data and predictive model described in this report.

Alongside these deployment extensions, further engineering work would be required to confirm that the pipeline scales efficiently to fleets considerably larger than the one hundred engines studied here, an important consideration given that a large airline or lessor may need to plan maintenance across several thousand engines simultaneously.

Each of these directions would extend the work presented in this report along a different dimension, whether toward harder benchmark conditions, more expressive treatment of predictive uncertainty, richer modelling and optimization techniques, or a more complete operational deployment, and each represents a natural and realistic next step building on the prediction to scheduling pipeline whose design, implementation, and verification have been the subject of this report.

References

The following references correspond to the sources cited throughout this report, reproduced here in the order in which they first appear in the text.

- [1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, Damage propagation modeling for aircraft engine run to failure simulation, in International Conference on Prognostics and Health Management (PHM), 2008, pages 1 to 9.
- [2] A. Saxena and K. Goebel, Turbofan Engine Degradation Simulation Data Set (CMAPSS), NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, California, 2008.
- [3] I. de Pater, A. Reijns, and M. Mitici, Alarm based predictive maintenance scheduling for aircraft engines with imperfect Remaining Useful Life prognostics, Reliability Engineering and System Safety, volume 221, article 108341, 2022.
- [4] T. Chen and C. Guestrin, XGBoost: A Scalable Tree Boosting System, in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pages 785 to 794.
- [5] Q. Huangfu and J. A. J. Hall, Parallelizing the dual revised simplex method, Mathematical Programming Computation, volume 10, number 1, pages 119 to 142, 2018.
- [6] I. Singh, E. Popova, and E. Kee, Optimal Preventive Maintenance Policy Under a Budget Constraint, in Proceedings of the 16th International Conference on Nuclear Engineering, volume 1, Orlando, Florida, 2008, pages 13 to 18.
- [7] J. Sciau, A. Goyon, A. Sarazin, J. Bascans, C. Prud'homme, and X. Lorca, Using constraint programming to address the operational aircraft line maintenance scheduling problem, Journal of Air Transport Management, volume 115, 2024.